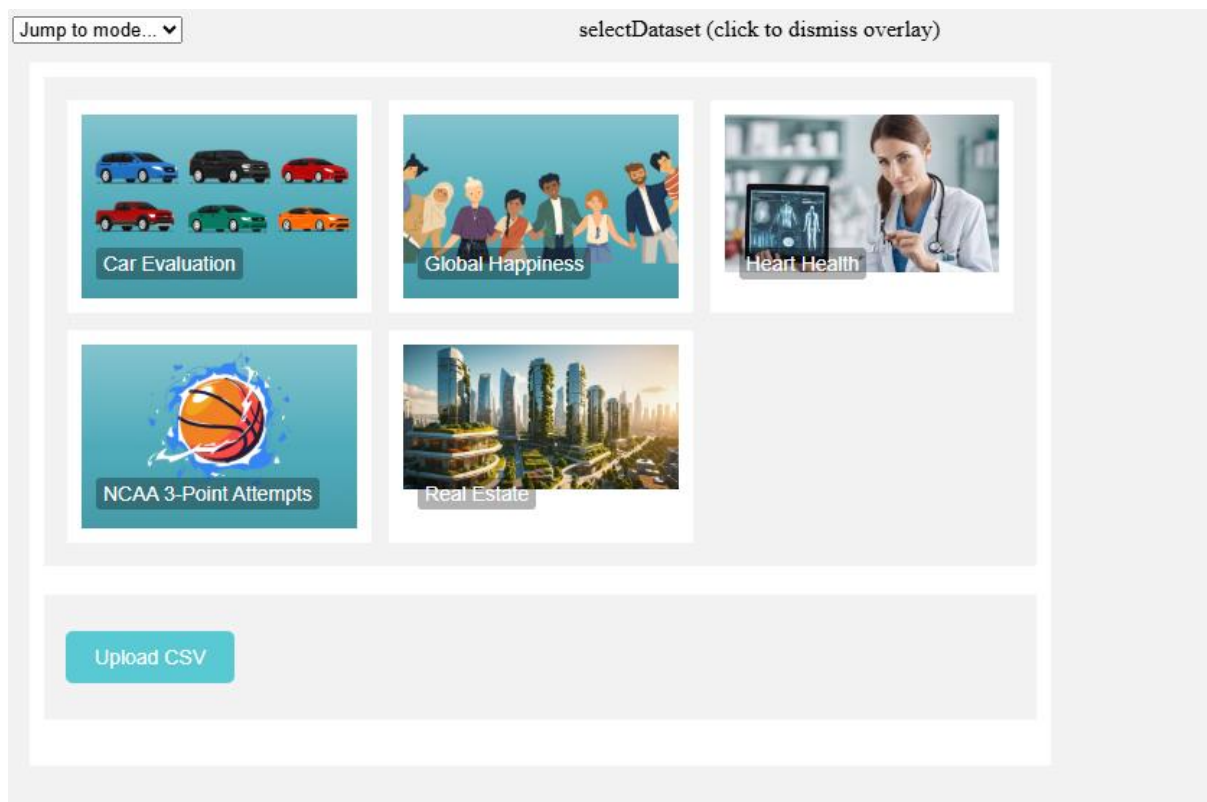


ML-Explorer

Dossier projet

Titre RNCP 5 Développeur web et web mobile
(DWWM)

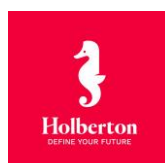


Edouard Halimi

19/12/2024

Table des matières

Table des matières	2
1. Introduction.....	4
2. Liste de compétences mises en œuvre.....	5
2.1 Compétences techniques	5
2.2 Compétences transversales.....	5
3. Expression des besoins du projet	5
3.1 Contexte et objectif.....	5
3.2 Besoins fonctionnels.....	7
3.2.1 Inscription et gestion des comptes utilisateurs	7
3.2.2 Gestion des datasets	7
3.2.3 Recherche et navigation	7
3.2.4 Diagramme du cheminement de l'application	8
3.3 Besoins non fonctionnels	9
3.4 Contraintes du projet	9
3.5 Critères de réussite.....	9
3.6 Organisation et Planning	10
4 Environnement technique.....	10
4.1 Langages de programmation utilisés / Frameworks, bibliothèques et base de données.....	10
4.1 Outils de développement	13
4.2 Plateformes ou services utilisés	13
5 Réalisations permettant la mise en œuvre des compétences	14
5.1 Réalisation du front-end.....	14
5.1.1 Présentation de maquettes de l'application, adaptation web et web mobile	14
5.1.2 Schéma de l'enchaînement des maquettes de l'application	16
5.1.3 Captures d'écran d'interfaces utilisateur, adaptation web et web mobile.....	17
5.1.4 Initialisation du projet	19
5.1.5 Installation des dépendances.....	19
5.1.6 Structure du front-end.....	20
5.1.7 Les composants	20
5.1.8 Configuration des routes	23
5.1.9 Responsive design	25
5.1.10 Les tests	25
5.1.11 Le déploiement.....	27
5.2 Réalisation du back-end.....	28
5.2.1 Initialisation du projet	29
19/12/2024	2



5.2.2	Installation des dépendances.....	29
5.2.3	Configuration du serveur Express.....	29
5.2.4	Présentation de la base de données.....	30
5.2.5	Création des Routes et Contrôleurs	35
5.2.6	Création des middlewares.....	35
5.2.7	Extraits de code	36
5.2.8	Test de l'API.....	38
5.3	Présentation d'éléments de sécurité de l'application	39
5.3.1	Utilisation de variable d'environnement	39
5.3.2	Configuration CORS.....	41
5.3.3	Sécurités lors de la création d'un compte utilisateur.....	41
5.3.4	Authentification via Json Web Token (JWT).....	42
5.4	Présentation des tests.....	43
5.5	Description de la veille, vulnérabilités de sécurité.....	45
6	Conclusion	45
7	Annexes	47

1. Introduction

Lorsque je me suis intéressé au machine learning, j'ai commencé par m'exercer sur deux plateformes : Kaggle, qui est plutôt orientée data science, et Hugging Face, qui est axée sur le développement de modèles d'IA. En m'exerçant, j'ai appris les notions de base sur le fonctionnement des modèles d'Intelligence Artificielle, comme les réseaux de neurones, l'apprentissage supervisé et non supervisé, ainsi que la validation croisée des modèles. J'ai alors réalisé qu'il serait utile de proposer un produit interactif permettant aux utilisateurs n'ayant pas ou peu de notions de s'initier aux fondamentaux de l'Intelligence Artificielle. Ce module éducatif viserait à leur expliquer de manière ludique et accessible les concepts essentiels, avant qu'ils ne se lancent sur des plateformes plus spécialisées comme Kaggle ou Hugging Face. En partant de cette idée, j'ai décidé d'en faire mon projet de soutenance : **ML-Explorer**, une application interactive visant à rendre l'apprentissage du machine learning plus simple et accessible. Ce projet offre non seulement un parcours d'apprentissage progressif, mais aussi une série de démonstrations interactives permettant aux utilisateurs de visualiser et de comprendre le comportement des différents modèles. Mon objectif était de créer une expérience immersive qui motive les débutants à se familiariser avec ces concepts avant de s'aventurer dans des environnements plus complexes.

J'ai travaillé seul sur ce projet, mais j'ai régulièrement échangé avec mes camarades de promotion (C23), car nous faisons face à des problématiques de développement similaires.

2. Liste de compétences mises en œuvre

2.1 Compétences techniques

Développer la partie front-end d'une application web ou web mobile sécurisée :

- Installer et configurer son environnement de travail en fonction du projet web ou web mobile.
- Maquetter des interfaces utilisateur web ou web mobile.
- Réaliser des interfaces utilisateur statiques web ou web mobile.
- Développer la partie dynamique des interfaces utilisateur web ou web mobile.

Développer la partie back-end d'une application web ou web mobile sécurisée :

- Mettre en place une base de données relationnelle.
- Développer des composants d'accès aux données SQL.
- Développer des composants métier côté serveur.
- Documenter le déploiement d'une application dynamique web ou web mobile.

2.2 Compétences transversales

- Méthodologie Agile : Ecoute, Entraide, Coopération.
- Mettre en œuvre une démarche de résolution de problèmes.
- Apprentissage continu et en autonomie.
- Capacité de sélectionner et d'exploiter les ressources web.

3. Expression des besoins du projet

3.1 Contexte et objectif

Le projet consiste à développer une application web pour des étudiants ou simplement des personnes curieuses, qui souhaiteraient mieux comprendre comment fonctionne l'intelligence artificielle.

Le Machine Learning (ML), ou apprentissage automatique, est une branche de l'intelligence artificielle qui consiste à permettre à une machine d'apprendre à partir de données sans être explicitement programmée. Il existe plusieurs concepts clés en ML. Nous allons explorer certains d'entre eux :

- Apprentissage supervisé

L'apprentissage supervisé est une méthode d'apprentissage où le modèle est entraîné à partir d'un ensemble de données qui contient à la fois des caractéristiques d'entrée et les réponses désirées (étiquettes). L'objectif est de permettre au modèle de généraliser à partir de ces données pour faire des prédictions précises sur de nouvelles données. Il existe plusieurs types de tâches d'apprentissage supervisé, notamment la classification et la régression.

Exemple : Prédiction du prix de l'immobilier On pourrait avoir un jeu de données d'annonces immobilières qui inclut les caractéristiques de chaque maison (surface, nombre de pièces, emplacement, etc.) et le prix réel de la maison. Un algorithme d'apprentissage supervisé peut être entraîné sur ces données pour prédire le prix d'autres maisons à partir de leurs caractéristiques.

- Apprentissage non supervisé

On utilise l'apprentissage non supervisé lorsque nous disposons d'un ensemble de données d'entrée sans étiquettes correspondantes. L'objectif est de permettre au modèle de trouver la structure ou les motifs dans ces données. Les méthodes d'apprentissage non supervisé comprennent le clustering et la réduction de la dimensionnalité.

Exemple : Segmentation de la clientèle Supposons que nous ayons un jeu de données de clients d'un supermarché, avec des informations sur leurs transactions passées, mais sans aucune étiquette qui classe les clients en différents groupes. On peut utiliser un algorithme de clustering pour segmenter les clients en groupes distincts en fonction de leurs habitudes d'achat.

-Apprentissage par renforcement

Dans l'apprentissage par renforcement, un agent apprend à effectuer certaines actions dans un environnement afin de maximiser la récompense cumulative. Contrairement à l'apprentissage supervisé et non supervisé, il n'y a pas de réponse correcte ou incorrecte. L'apprentissage est basé sur l'expérimentation et la découverte.

Exemple : Un apprentissage par renforcement peut être utilisé pour entraîner un agent à jouer à des jeux comme l'échec. L'agent, initialement, ne connaît pas la stratégie de jeu mais il apprend par l'expérience. Chaque bon mouvement est récompensé et en conséquence, l'agent apprend à faire des mouvements qui augmentent sa chance de victoire.

L'objectif principal est d'initier aux fondamentaux du machine learning grâce à une interface didactique et ludique.

3.2 Besoins fonctionnels

3.2.1 Inscription et gestion des comptes utilisateurs

Inscription et connexion des utilisateurs :

Système de création de comptes utilisateurs, chaque utilisateur correspond à un email et un mot de passe. Le hachage du mot de passe est assuré par bcrypt, et lorsque l'utilisateur est créé, un token JWT est généré.

Le composant « Register » vérifie la réponse de la requête d'enregistrement et redirige l'utilisateur vers la page de connexion si l'enregistrement est réussi.

Système de connexion des utilisateurs :

Système de connexion pour les utilisateurs, avec un point de terminaison qui vérifie si l'utilisateur existe, s'il existe vérifie le mot de passe, et si le mot de passe est correct, génère un token JWT.

Profil Utilisateur :

Chaque utilisateur doit pouvoir gérer son profil, mettre à jour ses informations ou pouvoir supprimer entièrement son compte.

3.2.2 Gestion des datasets

Pour ajouter un nouvel ensemble de données, vous avez besoin de trois fichiers : .csv, .json et .jpg. Après les avoir ajoutés au bon dossier, mettez à jour le fichier .json avec les détails de l'ensemble de données. Testez l'ensemble de données en vérifiant si les métadonnées des colonnes s'affichent correctement dans le panneau de l'application.

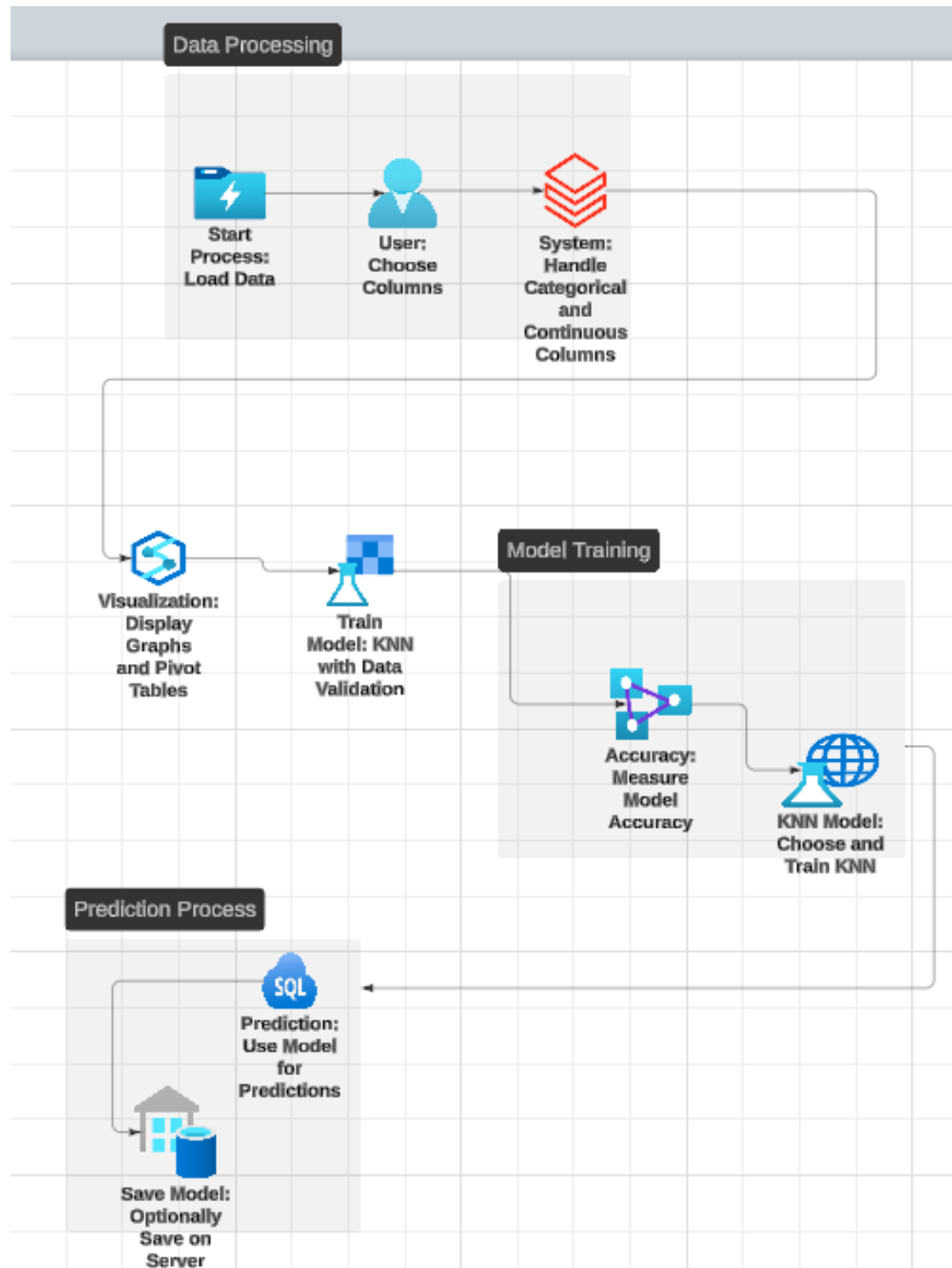
Veillez à ce que chaque colonne soit correctement répertoriée dans le fichier .json et testez son fonctionnement dans l'outil en mettant en surbrillance chaque colonne et en vous assurant que ses métadonnées s'affichent correctement dans le panneau de droite.

3.2.3 Recherche et navigation

L'application utilise une approche scène par scène, avec une logique centralisée pour contrôler le moment où les boutons « Précédent » et « Suivant » sont affichés et activés.

Par exemple, le bouton « Suivant » ne fonctionnera pas tant que l'utilisateur n'aura pas sélectionné une étiquette.

3.2.4 Diagramme du cheminement de l'application



3.3 Besoins non fonctionnels

Sécurité des données : Protection et confidentialité des informations personnelles des utilisateurs

Performance : Optimisation des ressources et du code pour assurer une navigation fluide

Interface Utilisateur : Navigation intuitive et structurée logiquement

Accessibilité : Fiabilité du déploiement et de l'hébergement

3.4 Contraintes du projet

Dans le but d'apprendre des technologies à fort potentiel, j'ai décidé de développer mon site en utilisant des langages et frameworks que je ne connaissais pas ou peu.

J'ai opté en front-end pour du Javascript / React.js, technologies très utilisées par les développeurs du monde entier. Ce choix me semblait évident dans le contexte web actuel et répondait aux nécessités de mon projet.

Pour le back-end, j'ai choisi Node.js et MongoDB, dans le but de répondre à l'architecture MERN (MongoDB, Express, React, Node), ceci afin d'optimiser les performances de mon application, et permettre une meilleure compatibilité des différents composants.

3.5 Critères de réussite

Lors de la définition des axes du projet, je me suis fixé 4 critères de réussite :

Expérience Utilisateur Intuitive

- Interface simple et progressive par étapes
- Instructions dynamiques adaptées à chaque scène
- Navigation claire avec boutons "Précédent" et "Suivant" contextuels

Flexibilité d'Exploration des Données

- Capacité de charger différents types de jeux de données
- Support des données catégorielles et continues
- Visualisations variées (graphiques, tableaux croisés)

Pédagogie de la Modélisation Machine Learning

- Démonstration pas à pas du processus d'apprentissage
- Visualisation de la performance du modèle
- Comparaison explicite des prédictions

Adaptabilité Technique

- Design responsive (mobile, tablette, ordinateur)
- Compatibilités multi-supports
- Possibilité de sauvegarder et réutiliser des modèles

3.6 Organisation et Planning

J'ai utilisé Trello pour organiser mon travail :

La première phase qui a duré 2 semaines entre le 01/09/24 et le 13/09/24, était un travail sur la Documentation et les Ressources en vue de la réalisation du projet (voir **annexe 16**)

La seconde phase qui a duré 1 semaine entre le 16/09/24 et le 20/09/24, avait pour but de réaliser les diagrammes et schémas nécessaires à la réalisation de mon application (voir **annexe 17**)

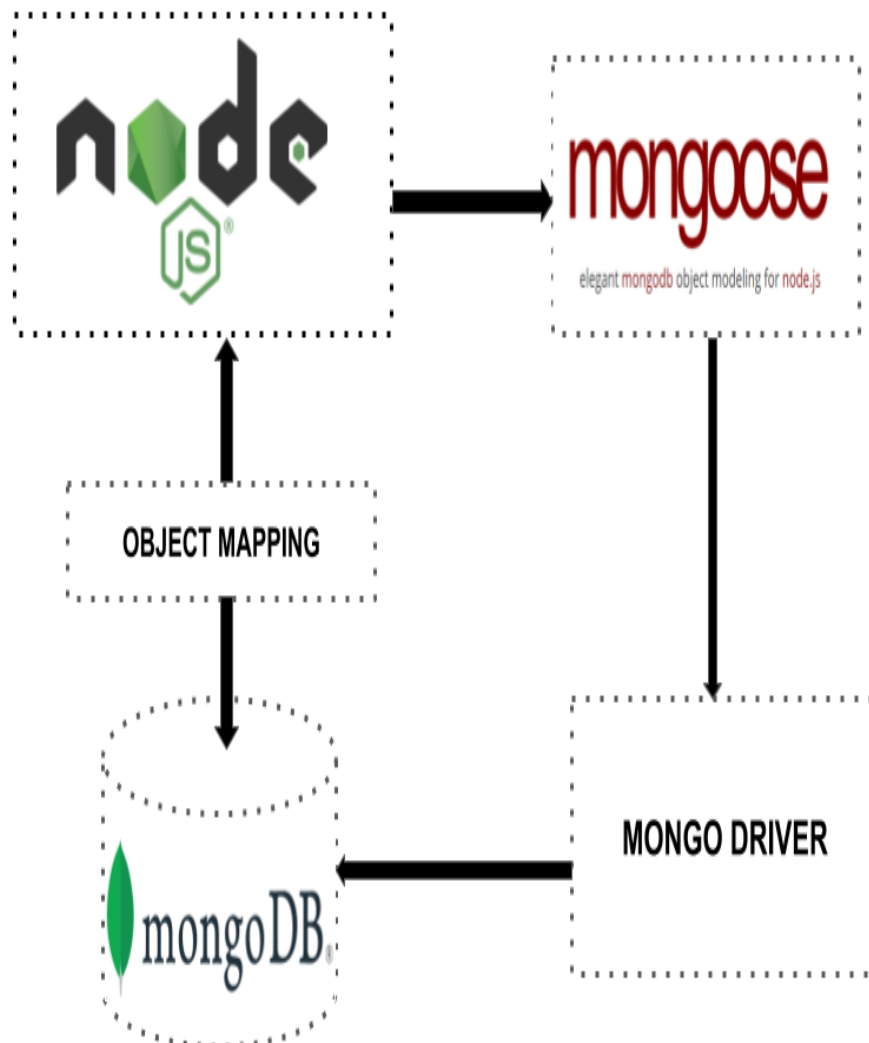
La troisième phase qui a duré 7 semaines entre le 23/09/24 et le 08/11/24 avait comme objectif de coder entièrement l'application (voir **annexe 18**)

4 Environnement technique

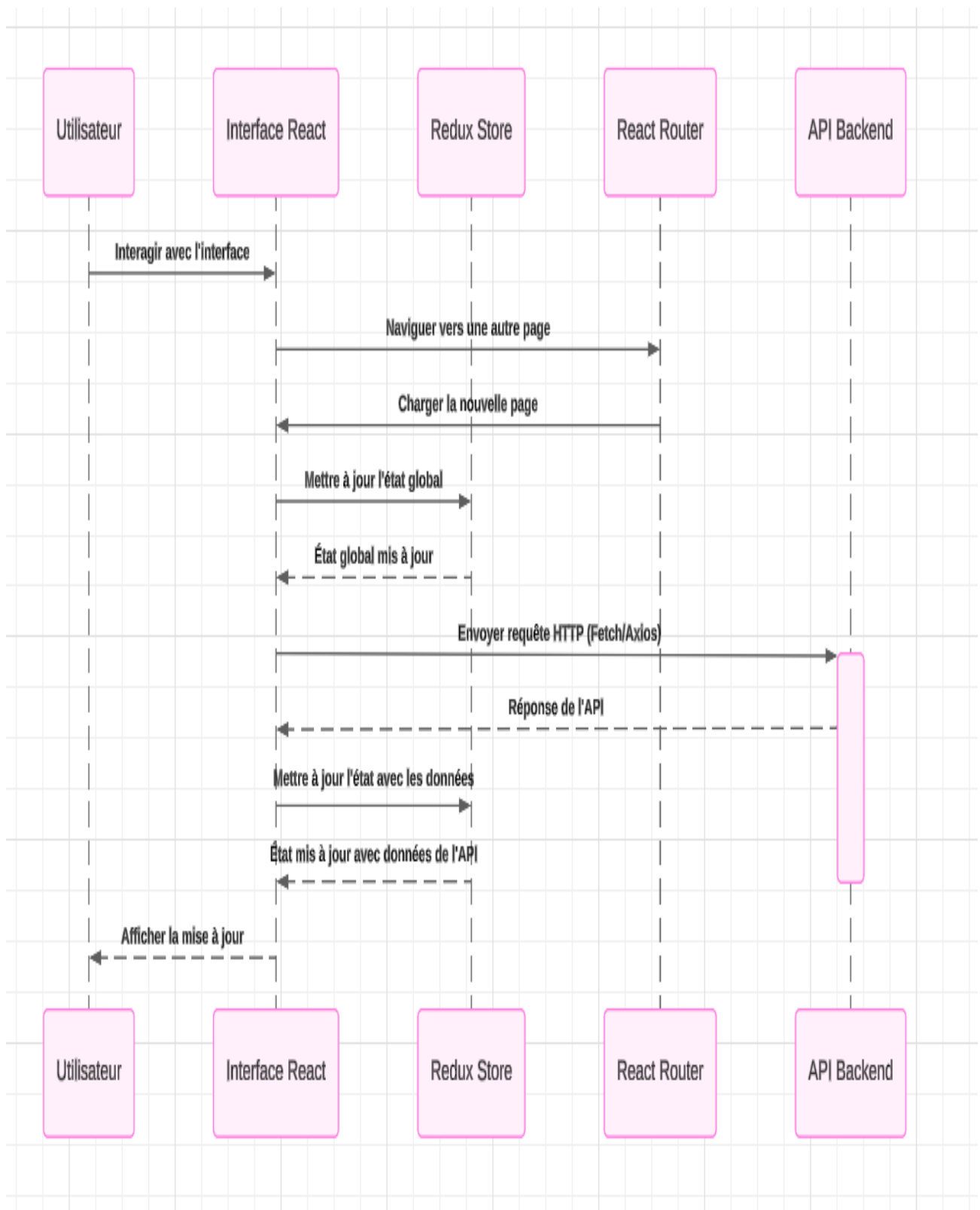
4.1 Langages de programmation utilisés / Frameworks, bibliothèques et base de données

J'ai utilisé principalement Javascript pour le développement de mon application, dans un cadre React pour le front et Node pour le back.

Voici une illustration du fonctionnement back-end de mon application



Et voici un diagramme du fonctionnement de mon application



4.1 Outils de développement

Bibliothèques

- React: Une bibliothèque JavaScript pour la construction d'interface utilisateur interactive.
- react-dom: Un connecteur entre React et le DOM.
- Redux: Une bibliothèque JavaScript qui aide à gérer l'état de l'application.
- Mongoose: Une bibliothèque qui facilite la communication entre MongoDB et Node.js.

Outils de développement

- react-dom: Un connecteur entre React et le DOM (Document Object Model).
- Babel-loader: Un package pour webpack qui transforme JavaScript basé sur la configuration Babel.
- Express: Un framework pour créer une application web sur le serveur, basé sur Node.js.
- Webpack: Un assembleur de modules pour JavaScript.
- Webpack Dev Server: Fournit un serveur de développement qui offre un rafraîchissement automatique des pages pour webpack.
- Babel-core: Le compilateur Babel qui possède une API pour l'utilisation des plugins, la transformation du code et la génération du code.
- React-redux: La liaison officielle de Redux pour React.
- ESLint: Un outil de linting pour JavaScript.
- Jest: Un framework de test pour JavaScript.
- Dotenv: Un module sans dépendance qui charge les variables d'environnement d'un fichier .env dans process .env.
- Axios: Un package client HTTP pour le navigateur et Node.js.

4.2 Plateformes ou services utilisés

1. GitHub : Je l'utilise en tant que plateforme d'hébergement de code source offrant non seulement les fonctionnalités de Git pour le contrôle de version, mais aussi ses propres caractéristiques.

2. GitHub Actions : C'est un outil précieux que j'emploie pour l'automatisation des workflows. Il me permet de créer, tester et déployer mon code directement depuis mon dépôt GitHub.
3. MongoDB Atlas : C'est ma solution pour une base de données dans le cloud. Elle me donne la flexibilité de contrôler où déployer mes données pour répondre aux exigences de conformité, de latence et de performance.
4. AWS (Amazon Web Services) : J'utilise AWS pour le stockage de mes bases de données. C'est une plateforme robuste de cloud computing.

5 Réalisations permettant la mise en œuvre des compétences

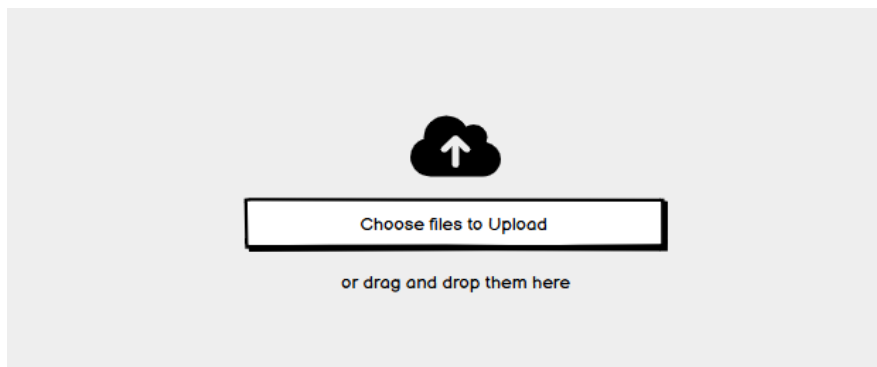
5.1 Réalisation du front-end

J'ai choisi le cadre de travail React pour le développement front end de mon application.

5.1.1 Présentation de maquettes de l'application, adaptation web et web mobile

Lors de la conception du projet, j'avais en tête 4 scènes principales dans mon application :

La première scène correspondait à la sélection de son dataset :



La deuxième scène correspondait au tableau du dataset sélectionné et à la construction de son modèle : c'est-à-dire choisir la colonne à prédire, en fonction d'une ou plusieurs autres colonnes correspondant aux « features », c'est-à-dire les datas que le modèle va utiliser lors de l'entraînement.

Products + Add Product

Filter: Type Vendor Material Color 11 Sort: Newest

Showing 30 results

#	Name	Type	Vendor	In Stock	Unit Cost	Material	Color
1	Street Pulse Hoodie	Hoodie	StreetGear	150	\$45	Cotton	Black
2	Urban Shadow Jacket	Jacket	CityWear	100	\$60	Polyester	Gray
3	Neon Splash Tee	T-Shirt	UrbanTrend	200	\$20	Cotton	Neon Green
4	Night Walker Jeans	Jeans	DenimWorks	80	\$50	Denim	Dark Blue
5	Retro High Sneakers	Sneakers	StreetFoot	8	\$90	Leather	White
6	Cityscape Cap	Cap	HeadTops	160	\$25	Cotton	Black
7	Skyline Socks	Socks	SockVault	300	\$10	Cotton	Multi
8	Moonlight Scarf	Scarf	WrapUp	150	\$30	Wool	Dark Gray
9	Starlight Belt	Belt	Accessorizeit	90	\$35	Leather	Black
10	Dawn Breaker Shorts	Shorts	SummerWaves	85	\$40	Cotton	Navy Blue
11	Glitch Graphic Tee	T-Shirt	PrintCraze	200	\$25	Cotton	White
12	Phantom Bag	Backpack	BagBasics	95	\$55	Nylon	Black
13	Drift Tank Top	Tank Top	CoreFit	180	\$18	Cotton	White
14	Vaporwave Hoodie	Hoodie	HypeStreet	130	\$48	Cotton	Pastel Pink
15	Troblazer Watch	Watch	TimeTrend	70	\$110	Metal	Silver
16	Comet Sneakers	Sneakers	StreetFoot	110	\$95	Leather	Red
17	Echo Leggings	Leggings	FitFashion	100	\$35	Spandex	Black
18	Prism Hat	Hat	HatHaven	165	\$28	Wool	Blue
19	Twilight Parka	Parka	NorthernWear	60	\$85	Polyester	Black
20	Sundown Gloves	Gloves	ChillGear	200	\$22	Leather	Gray
21	Cosmic Sweatpants	Sweatpants	RelaxMode	150	\$38	Cotton	Maroon
22	Sunset Fit Dress	Fit Dress	DressWorks	90	\$45	Cotton	Orange

La troisième scène correspondait au résultat du modèle donné, à savoir le taux de précision avec lequel le modèle était capable de prédire la colonne en question :

results (no overlay)

Result	Accuracy
Predict chest pain type based on fasting blood sugar (mmol/L) , resting blood pressure , maximum heart rate achieved	70.97% Details

Previous results

Previous results	Accuracy
Predict chest pain type based on fasting blood sugar (mmol/L) , resting blood pressure , maximum heart rate achieved	70.97%

Enfin, la quatrième scène correspondait à une fiche récapitulative permettant de sauvegarder son modèle et de décrire ce qui fonctionne bien ainsi que les limites de ce dernier.

Model name (required)

Intended Use
Describe the problem you think this model could help solve, or one potential app someone could make with this model.

Write a brief description.

Limitations and Warnings
Describe any limitations in how this model was created or how it should be used. You may say things like "Avoid using this model for..." or "Be cautious about...". Important questions to consider are:

- Does the data represent all possible users and scenarios?
- Did you gather enough data to be confident in the model's accuracy?
- Are there situations where this model definitely shouldn't be used?

Write a brief description.

About the Data
A dataset of test results for heart disease patients, used to predict whether or not someone may have heart disease.

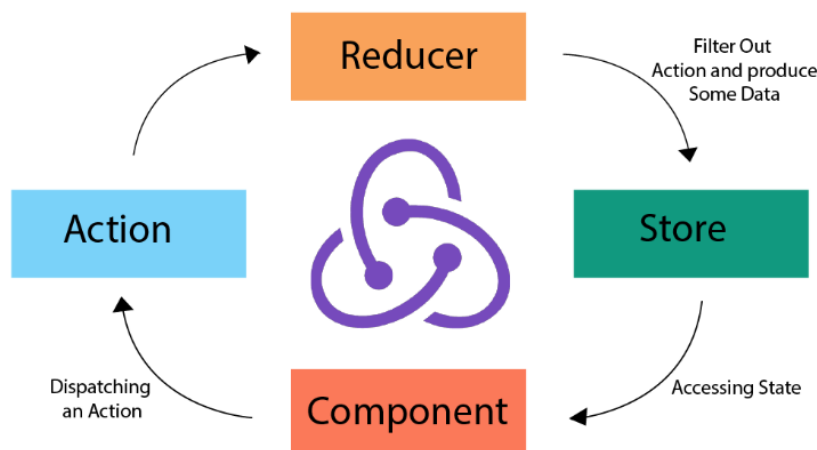
Column Descriptions (4)

J'ai par la suite rajouté une scène entre la scène 2 et la scène 3, qui correspond à une cinématique qui illustre l'entraînement du modèle. Cette approche visuelle a pour objectif de maintenir l'attention et l'intérêt de l'utilisateur, et de rendre l'application plus attractive.

5.1.2 Schéma de l'enchaînement des maquettes de l'application

J'ai rapidement esquissé les 4 principales scènes en utilisant Balsamiq (extrait ci-dessus), puis j'ai approfondi le travail au fil du développement, notamment en incorporant des éléments de style CSS.

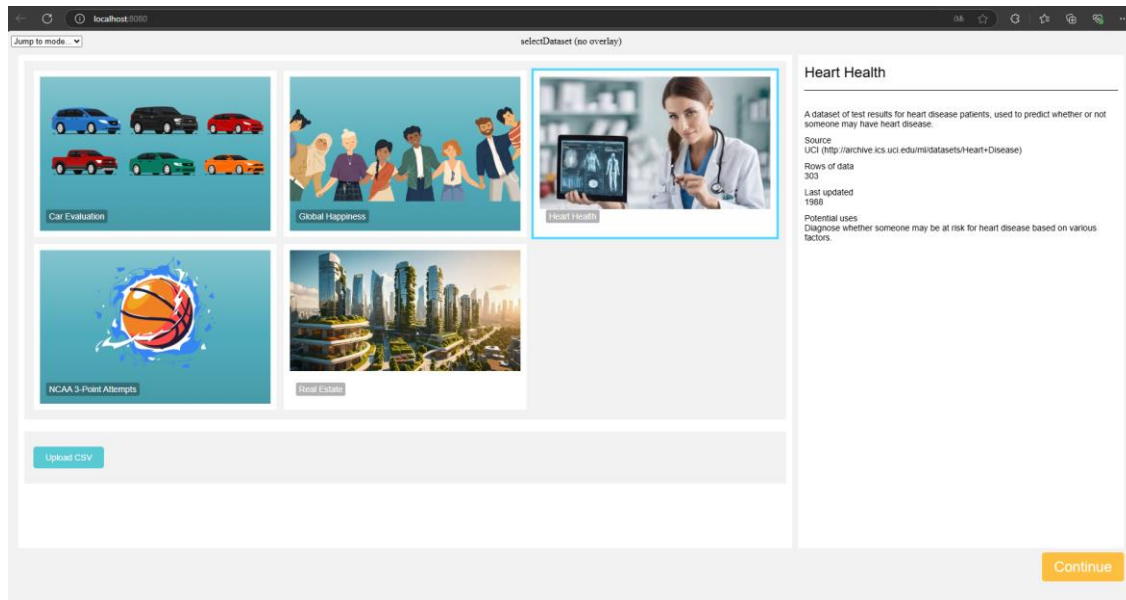
Enfin, j'ai intégré la navigation entre les scènes grâce à redux :



5.1.3 Captures d'écran d'interfaces utilisateur, adaptation web et web mobile

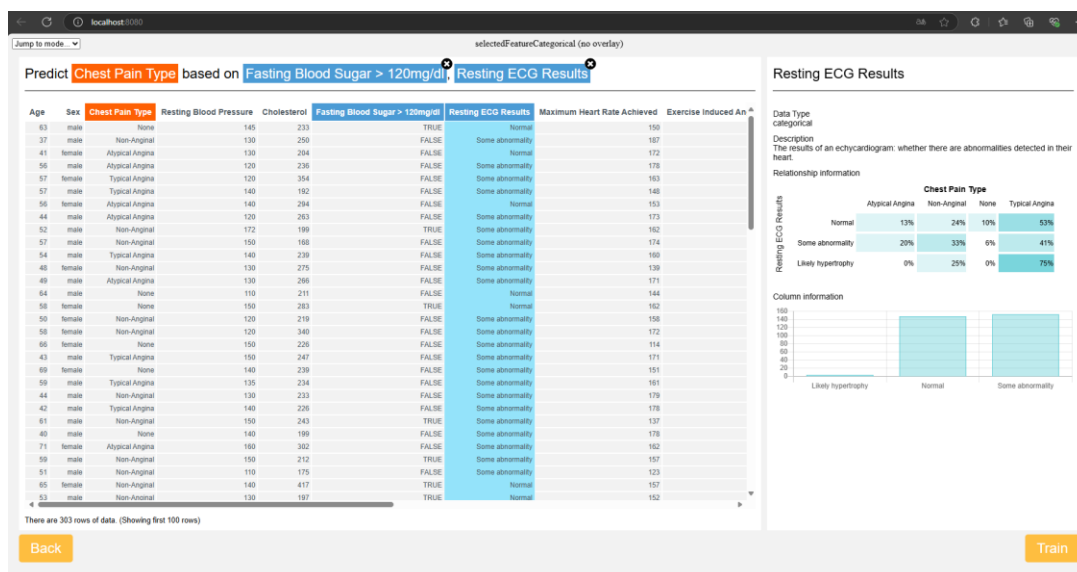
Voici les principales scènes de mon application :

Scène n°1 Chargement du dataset :



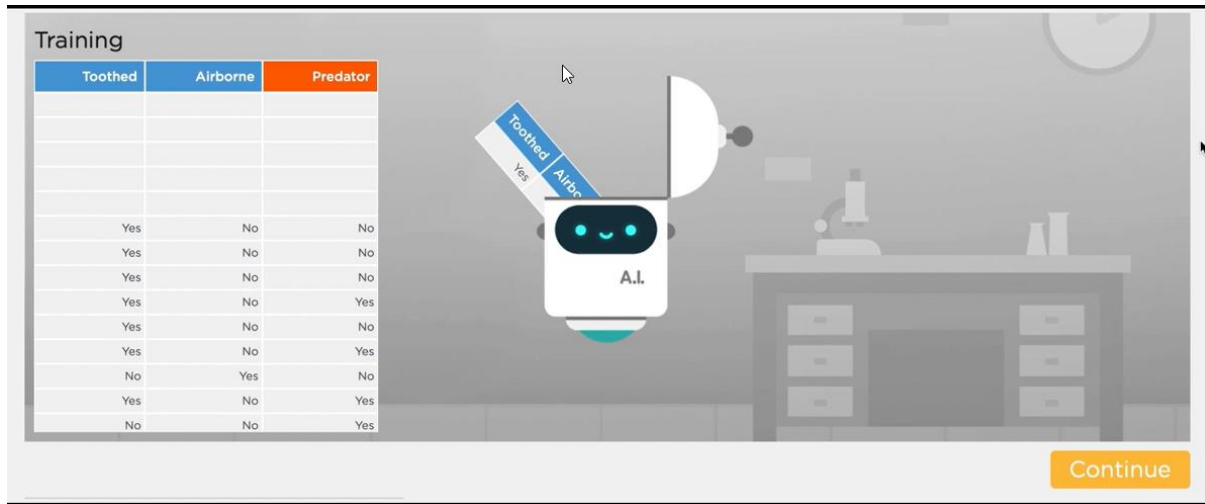
Ici, l'utilisateur peut choisir un dataset parmi la sélection, ou télécharger son propre dataset grâce au bouton en bas à gauche « Upload CSV »

Scène n°2 Choix de la prédiction « x » basé sur « y, z ... »



Dans cette scène, l'utilisateur accède à son dataset, choisit la colonne à prédire en orange, appuie sur le bouton « select label », puis sur le bouton « continue », puis choisit plusieurs autres colonnes, appuie sur « add feature », et enfin appuie sur « train » pour l'entraînement du modèle.

Scène n°3 Illustration visuelle de l'entraînement du modèle :



Dans cette scène, l'utilisateur observe le modèle « s'entraîner », à l'aide d'une représentation visuelle d'un petit robot qui « absorbe » les différentes données.

Scène n°4 Document pour sauvegarder son modèle :

Jump to mode...

saveModel (no overlay)

Predict

Chest Pain Type

based on

Fasting Blood Sugar > 120mg/dl

Resting ECG Results

Model name (required)

Intended Use

Describe the problem you think this model could help solve, or one potential app someone could make with this model.

Write a brief description.

Limitations and Warnings

Describe any limitations in how this model was created or how it should be used. You may say things like "Avoid using this model for..." or "Be cautious about...". Important questions to consider are:

- Does the data represent all possible users and scenarios?
- Did you gather enough data to be confident in the model's accuracy?
- Are there situations where this model definitely shouldn't be used?

Write a brief description.

About the Data

A dataset of test results for heart disease patients, used to predict whether or not someone may have heart disease.

Column Descriptions (3)

Back

Save

Enfin, l'utilisateur, peut sauvegarder son modèle, en remplissant les champs suivants :

- Model name
- Intended Use
- Limitations and Warnings

5.1.4 Initialisation du projet

Lors de l'initialisation du projet, j'ai d'abord créé un nouveau dossier pour mon projet.

- J'ai installé Node.js.
- J'ai installé React et ReactDOM.
- J'ai installé les autres packages nécessaires, tels que Redux, Babel Loader, Express et Mongoose.
- J'ai créé et configuré mon fichier webpack.config.js pour utiliser Babel et transpiler mon code React.
- J'ai configuré mon serveur Express dans un fichier séparé, server.js.
- J'ai configuré des scripts dans mon fichier package.json pour démarrer, tester et construire mon application.

5.1.5 Installation des dépendances

Les dépendances front-end pour ce projet sont :

- [react@18.3.1](#): Il s'agit de la bibliothèque ReactJS, qui est le cœur de notre application front-end.
- [react-dom@18.3.1](#): Ce package sert à faire le rendu des composants React dans le DOM.
- [redux@5.0.1](#): Redux est utilisé pour gérer l'état global de l'application.
- [react-redux@9.1.2](#): Ce package lie ensemble React et Redux, facilitant l'utilisation de Redux dans un projet React.
- [@testing-library/react](#): bibliothèque qui fournit un ensemble de fonctions pour interagir avec des composants React lors de tests, de manière similaire à ce que ferait un utilisateur réel.
- [@testing-library/jest-dom](#): Offre un ensemble de matchers personnalisés jest-dom pour écrire des assertions plus facilement lisibles et expressives.
- [@fortawesome/react-fontawesome@0.1.7](#) et [@fortawesome/free-solid-svg-icons@5.11.2](#): Ces deux packages sont utilisés pour utiliser les icônes de FontAwesome dans notre application React.
- [chart.js@2.9.4](#) et [react-chartjs-2@2.11.1](#): Ces deux librairies sont utilisées pour la visualisation de données sous forme graphiques.

- [query-string@4.1.0](#): Cette librairie est utilisée pour parser les chaînes de requête dans les URL.

5.1.6 Structure du front-end

Le front-end est basé sur React et utilise la structure standard des applications React. Il y a un composant racine qui rend d'autres composants en fonction de l'état et des routes. Ces composants sont organisés en dossiers, chaque dossier contenant le code du composant et son style. Redux est utilisé pour gérer l'état global, avec actions et réducteurs séparés. Les composants peuvent faire des appels API via Axios. FontAwesome et Chart.js sont utilisés pour l'UI.

Palette de couleurs

J'ai imaginé une palette de couleurs minimale et moderne qui reflète l'esprit de l'analyse de données. J'ai utilisé des tons de gris et orange pour représenter la confiance et l'intelligence, et des gris doux pour les arrière-plans pour un contraste minimal.

- Orange léger #FFA500 : Je l'ai utilisé pour les boutons et les liens importants
- Noir #020000 : Il a été utilisé pour les en-têtes et le texte de grande taille
- Gris clair #CCD1D1 : Utilisé pour les arrière-plans et les zones de contenu

Police de caractères

J'ai considéré une police Aptos pour refléter la modernité et la lisibilité, pour correspondre à la nature du projet :

SEO

J'ai axé le SEO du projet sur le référencement technique en raison de la nature du projet ML-Explorer. J'ai veillé à ce que les titres et descriptions méta soient clairement définis pour chaque page, incluant des mots clés tels que "machine learning", "ML Explorer", "analyse de données".

5.1.7 Les composants

Voici les principaux composants React, présent dans le dossier UIComponents :

- **DataTable.jsx** : Ce composant est utilisé pour afficher les données importées dans un format de tableau. Il a plusieurs responsabilités : rendre les en-têtes de colonne, rendre les rangées de données, interaction utilisateur. (voir **annexe 1**)
- **DataDisplay.jsx**: Ce composant affiche les données importées à l'utilisateur. Si aucune donnée n'a été importée, il ne rend rien. Sinon, il affiche le nombre de lignes de données et un composant DataTable qui rend les données elles-mêmes. (voir **annexe 2**)
- **DataCard.jsx**: Ce composant affiche les informations détaillées du jeu de données sélectionné. Il affiche des informations comme le nom du jeu de données, sa description, le nombre de lignes, etc. (voir **annexe 3**)
- **AddFeatureButton.jsx**: Ce bouton permet à l'utilisateur d'ajouter une colonne sélectionnée à la liste des fonctionnalités qui seront utilisées pour l'entraînement du modèle.
- **ColumnDataTypeDropdown.jsx**: Ce composant est un menu déroulant qui permet à l'utilisateur de choisir le type de données pour une colonne sélectionnée.
- **ColumnDetailsCategorical.jsx**: Ce composant affiche les détails d'une colonne catégorielle, y compris un histogramme des différentes catégories et leurs fréquences.
- **ColumnDetailsNumerical.jsx**: Ce composant affiche des informations détaillées sur une colonne de type numérique, telles que la gamme de valeurs et si toutes les entrées sont de type numérique.
- **ColumnInspector.jsx**: Ce composant inspecte une colonne sélectionnée, et affiche des détails pertinents en fonction du type de donnée de la colonne. Il peut aussi afficher un bouton pour ajouter la colonne en tant que caractéristique ou étiquette pour l'entraînement du modèle.
- **GenerateResults.jsx** : Ce composant gère les animations lors de la génération des résultats. Il utilise un intervalle pour mettre à jour une animation frame par frame et rend le contenu à différents états de l'animation.
- **ModelCard.jsx** : Ce composant affiche les détails de la carte du modèle, y compris le nom du modèle, l'exactitude, l'utilisation prévue, les limites, les détails du jeu de données, les caractéristiques et les labels. Ces détails sont utiles pour comprendre comment le modèle fonctionne et quels facteurs contribuent à ses prédictions. (voir **annexe 4**)
- **Predict.jsx** : Ce composant permet de prédire les résultats en entrée à l'aide du modèle entraîné. Il permet aux utilisateurs d'entrer les valeurs pour les

caractéristiques sélectionnées et rend la prédiction du label par le modèle. (voir **annexe 5**)

- **ResultsDetails.jsx**: Ce composant affiche les détails des résultats du modèle. Il comprend une table qui montre les prédictions correctes et incorrectes du modèle. (voir **annexe 6**)
- **ResultsToggle.jsx** : Ce composant est un bouton basculant qui permet aux utilisateurs de basculer entre les résultats corrects et incorrects.
- **ResultsTable.jsx** : Ce composant affiche un tableau récapitulatif des résultats de l'entraînement du modèle. Il met en évidence les entrées de données, les labels réels, et les labels prédits par le modèle d'A.I. Ce composant fournit un moyen visuel d'évaluer la performance du modèle d'A.I. (voir **annexe 7**)
- **SaveModel.jsx** : Ce composant gère la sauvegarde d'un modèle entraîné. Il permet aux utilisateurs de définir un nom pour le modèle, d'ajouter une description pour la dataset utilisée et d'inclure des informations sur les utilisations potentielles et les mésusages potentiels du modèle. Ce composant est important pour la gestion et le suivi des modèles d'A.I entraînés. (voir **annexe 8**)
- **ScatterPlot.jsx** : Ce composant crée un graphique de dispersion qui vous permet de visualiser les résultats de votre modèle d'A.I. Il utilise le package 'react-chartjs-2' pour générer ce graphique.
- **ScrollableContent.jsx** : Ce composant encapsule du contenu qui nécessite le défilement. Il est souvent utilisé pour afficher une liste d'objets ou d'informations qui pourraient ne pas tenir dans la zone d'affichage disponible.
- **SelectDataset.jsx** : Ce composant gère la sélection d'une dataset pour l'entraînement du modèle d'A.I. Il permet aux utilisateurs de choisir parmi plusieurs datasets prédéfinis ou de télécharger leur propre dataset au format CSV. Le choix de la dataset est une étape clé de l'entraînement du modèle d'A.I. (voir **annexe 9**)
- **SelectLabelButton.jsx** : Ce composant est un bouton qui, lorsqu'il est cliqué, définit une colonne spécifique de la dataset comme étant la colonne des labels. Les labels sont utilisés par le modèle d'A.I pour effectuer des prédictions; donc, choisir la bonne colonne comme label est crucial pour l'entraînement du modèle.
- **Statement.jsx** : Il gère l'affichage d'une déclaration qui résume le modèle actuel (label et caractéristiques sélectionnés). Il fournit également des fonctionnalités pour supprimer des features ou la colonne label via des icônes de suppression. Il reçoit des propriétés depuis le store Redux, y compris la colonne label actuelle, les features sélectionnées, et l'état de l'overlay d'instructions. Les méthodes `removeLabel()` et `removeFeature(id)` sont utilisées pour supprimer respectivement la colonne label et une feature spécifique.

- **TrainModel.jsx** : Il gère l'entraînement du modèle. Durant le processus d'entraînement, une animation est affichée où chaque ligne de la dataset se dirige vers la tête du robot AI. Le composant récupère l'état de Redux pour déterminer quand commencer et arrêter l'animation. Le tableau de données est affiché avec une opacité qui varie en fonction de l'état de l'animation. (voir **annexe 10**)

5.1.8 Configuration des routes

Dans cette application, je ne travaille pas avec la configuration de routage traditionnelle que vous pourriez voir avec react-router-dom. Au lieu de cela, je gère la navigation entre différents "écrans" ou "panels" par le biais des actions Redux et de l'état du store.

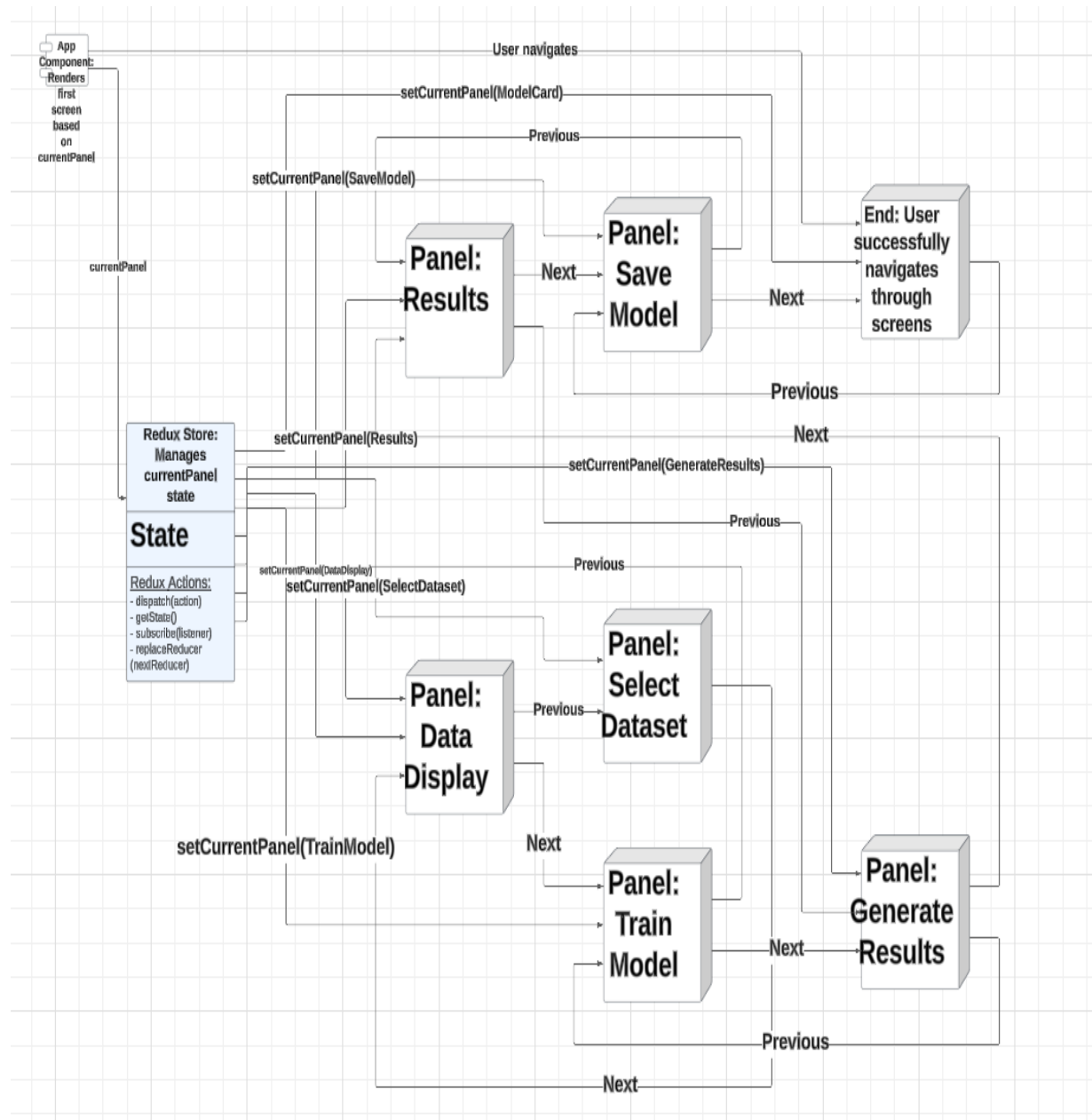
Voilà comment je le fais :

- J'ai plusieurs composants qui servent d'écrans différents dans l'application : SelectDataset, DataDisplay, TrainModel, GenerateResults, Results, SaveModel et ModelCard.
- L'écran actuellement affiché est déterminé par la propriété currentPanel dans le store Redux. Je mets à jour cette propriété avec l'action setCurrentPanel(panel).
- Dans le composant App, j'utilise la propriété currentPanel pour décider quel écran (ou panel) je dois afficher à l'utilisateur. Par exemple, si currentPanel vaut "selectDataset", alors je rends le composant SelectDataset, et ainsi de suite pour les autres écrans.
- Je gère les boutons de navigation (suivant et précédent) dans le composant PanelButtons. Lorsque ces boutons sont cliqués, ils déclenchent soit la fonction onClickPrev (pour le bouton précédent) ou onClickNext (pour le bouton suivant). Ces fonctions appellent ensuite setCurrentPanel avec le panel approprié à afficher.

- De cette manière, je gère le passage d'un écran à un autre en modifiant l'état `currentPanel` dans le store Redux, ce qui déclenche un nouveau rendu de mon application avec le bon panel affiché.

Il est important de noter que cette approche de gestion de la navigation est spécifique à cette application. Elle diffère de l'utilisation classique de `react-router-dom` pour le routage entre les pages. Elle convient mieux à une application de type "assistant" où l'utilisateur avance d'un écran à un autre dans un ordre spécifique.

Voici un diagramme qui représente la gestion des écrans via Redux :



5.1.9 Responsive design

L'application est réactive horizontalement et fonctionne bien en mode paysage sur mobile. Il s'ajuste également verticalement à l'aide de flexbox, certains éléments étant fixés en hauteur et d'autres réglables.

5.1.10 Les tests

Pour réaliser les tests, j'utilise la bibliothèque @testing-library/react pour rendre les composants et interagir avec eux, et jest pour exécuter les tests et faire des assertions. Ils testent que les composants se comportent correctement en réponse à des actions de l'utilisateur, comme des clics ou des changements de type de données de colonne, en vérifiant que les bonnes fonctions sont appelées avec les bonnes valeurs.

Voici 3 exemples de tests de mes composants react :

```
AddFeatureButton.test.js x
1  import React from 'react';
2  import { render, fireEvent } from '@testing-library/react';
3  import AddFeatureButton from './AddFeatureButton';
4
5  test('ajoute une colonne comme feature lors du clic', () => { new *
6    const addSelectedFeatureMock = jest.fn();
7    const { getByText } = render(
8      <AddFeatureButton column="TestColumn" addSelectedFeature={addSelectedFeatureMock} />
9    );
10   const button = getByText('ajoute une feature/i);
11
12   fireEvent.click(button);
13   expect(addSelectedFeatureMock).toHaveBeenCalledWith('TestColumn');
14 });
```

```
AnimationDescription.test.js x
1 import React from 'react';
2 import { render } from '@testing-library/react';
3 import AnimationDescription from './AnimationDescription';
4
5 test('affiche la bonne description', () :void => { new *
6   const { getByText : BoundFunction<<T=HTMLElement extends HTMLEle...> } = render(<AnimationDescription />);
7   const descriptionElement : (container: HTMLElement, id: M... = getByText( args: /Votre description d'animation ici/i);
8
9   expect(descriptionElement).toBeInTheDocument();
10 });
```

```
ColumnDataTypeDropdown.test.js x
1 import React from 'react';
2 import { render, fireEvent } from '@testing-library/react';
3 import ColumnDataTypeDropdown from './ColumnDataTypeDropdown';
4 import { ColumnTypes } from '../constants.js';
5
6 test('change le type de données de colonne', () :void => { new *
7   const setColumnsByDataTypeMock = jest.fn();
8   const { getByRole : BoundFunction<<T=HTMLElement extends HTMLEle...> } = render(
9     <ColumnDataTypeDropdown columnId="TestColumn" currentDataType={ColumnTypes.CATEGORICAL} setColumnsByDataType
10   );
11   const dropdown : (container: HTMLElement, role:... = getByRole( args: "combobox");
12
13   fireEvent.change(dropdown, options: { target: { value: ColumnTypes.CONTINUOUS } });
14   expect(setColumnsByDataTypeMock).toHaveBeenCalledWith( expected: "TestColumn", ColumnTypes.CONTINUOUS);
15 });
```

Et voici un tableau qui récapitule l'ensemble de ces tests :

scénarios de test	résultats attendus	résultats retournés
<u>AddFeatureButton.test.js:</u>		
Vérifie si la fonction addSelectedFeatureMock a été appelée avec l'argument "TestColumn" quand un clic est simulé sur le bouton "Ajouter une fonction"	La fonction addSelectedFeatureMock soit appelée avec l'argument "TestColumn"	"ajoute une colonne comme feature lors du clic"
<u>AnimationDescription.test.js:</u>		
Vérifie si l'élément "descriptionElement" est présent dans le document après le rendu du composant "AnimationDescription"	L'élément "descriptionElement" est présent dans le document	"affiche la bonne description"
<u>ColumnDataTypeDropdown.test.js:</u>		
Vérifie si la fonction setColumnsByDataTypeMock est appelée avec les arguments "TestColumn" et ColumnTypes.CONTINUOUS	setColumnsByDataTypeMock est appelée avec les arguments "TestColumn" et ColumnTypes.CONTINUOUS	"change le type de données de colonne"
<pre>Test Suites: 3 passed, 3 total Tests: 3 passed, 3 total Snapshots: 0 total Time: 1.515s, estimated 2s</pre>		

5.1.11 Le déploiement

Pour déployer le front-end de mon application, j'ai utilisé AWS. Voici comment j'ai procédé :

- J'ai créé un bucket AWS S3 : J'ai configuré un nouveau bucket S3 via la console AWS et autorisé l'accès public pour permettre à CloudFront de servir les fichiers.
- J'ai téléchargé les fichiers de build : J'ai exécuté npm run build pour construire le front-end, puis j'ai uploadé tous les fichiers de sortie dans le bucket S3 que j'ai créé précédemment. (voir **annexe 15**)
- J'ai mis en place AWS CloudFront : J'ai créé une nouvelle distribution CloudFront qui utilise le bucket S3 comme origine.
- J'ai pointé le domaine vers CloudFront : J'ai mis à jour les enregistrements DNS du domaine pour qu'ils pointent vers l'URL de CloudFront, rendant ainsi l'application accessible en ligne.

5.2 Réalisation du back-end

J'ai choisi le cadre de travail (framework) Node.js pour la réalisation du back-end de mon application.

J'ai choisis une architecture MVC (Modèle-Vue-Contrôleur), j'ai divisé l'application en trois composants principaux :

Modèle : J'ai représenté les données de l'application et les règles métier.

Vue : Je me suis occupé de la présentation des données (l'interface utilisateur).

Contrôleur : J'ai géré l'interaction entre la vue et le modèle.

Puis, j'ai structuré l'application ceci:

```
/ML-Explorer
|-- /model
|   |-- user.js
|   |-- dataset.js
|-- /view
|   |-- home.js
|   |-- login.js
|   |-- register.js
|-- /controller
|   |-- userController.js
|   |-- datasetController.js
|-- server.js
|-- package.json
```

5.2.1 Initialisation du projet

- Installation de Node.js et NPM : J'ai installé Node.js qui est l'environnement d'exécution utilisé pour exécuter mon serveur en JavaScript, et NPM qui est le gestionnaire de paquets de Node.js. Suite à l'installation de Node.js, NPM a été installé simultanément.
- Initialisation du projet : J'ai initialisé mon projet en exécutant la commande `npm init -y` dans l'invite de commandes. En conséquence, un fichier `package.json` a été créé dans le répertoire de mon projet. Ce fichier contient les informations relatives à mon projet et ses dépendances.

5.2.2 Installation des dépendances

- *express* : Il s'agit d'un framework pour Node.js qui est utilisé pour créer le serveur.
- *bcryptjs* : C'est une bibliothèque pour crypter des mots de passe.
- *jsonwebtoken* : C'est utilisé pour créer des tokens d'accès pour l'authentification.
- *dotenv* : C'est une bibliothèque qui permet de charger des variables d'environnement à partir d'un fichier `.env`.
- *mongoose* : C'est l'interface officielle de MongoDB pour Node.js. Je l'utilise pour interagir avec la base de données MongoDB.
- *mongoose* : C'est une bibliothèque d'Object Data Modelling (ODM) pour MongoDB et Node.js. Elle aide à gérer les relations entre les données et fournit des fonctionnalités de validation des schémas.
- *passport* et *passport-local* : Passport est un middleware d'authentification pour Node.js. Passport-local est un plugin pour Passport qui gère l'authentification avec nom d'utilisateur et mot de passe.
- *express-session* : Il s'agit d'un middleware pour gérer les sessions pour Express.
- *connect-flash* : Il s'agit d'un middleware qui fournit un moyen facile de flasher des messages à l'utilisateur, généralement après qu'une action ait été effectuée.
- *axios* : C'est une bibliothèque qui facilite les requêtes HTTP depuis Node.js.

5.2.3 Configuration du serveur Express

Voici comment j'ai configuré mon serveur Express dans mon fichier `server.js` (voir **Annexe 11**) :

- **Importation des dépendances** : Le fichier commence par l'importation de toutes les dépendances nécessaires telles que *express* (pour la configuration du serveur), *dotenv* (pour la gestion des variables d'environnement), *mongoose* (pour la connexion et la gestion de la base de données MongoDB), *passport* et *passport-local* (pour l'autorisation et l'authentification des utilisateurs), *bcryptjs* (pour le cryptage des mots de passe), *express-session* (pour gérer les sessions), *path* (pour gérer les chemins de fichiers), *jsonwebtoken* (pour gérer les tokens d'authentification) et *connect-flash* (pour afficher les messages flash).
- **Configuration de la base de données MongoDB** : Ensuite, une connexion est établie à la base de données MongoDB avec *mongoose.connect* en utilisant la chaîne de connexion provenant des variables d'environnement du fichier *.env*.
- **Création du schéma utilisateur**: Un nouveau schéma Mongoose est créé pour les utilisateurs, contenant un nom d'utilisateur (*username*) et un mot de passe (*password*).
- **Configuration de la session express** : La session express est configurée avec une clé secrète provenant du fichier *dotenv* (*process.env.SESSION_SECRET*). D'autres configurations comprennent *resave : false* (qui spécifie que la session ne doit pas être enregistrée si elle n'a pas été modifiée) et *saveUninitialized : false* (qui empêche de stocker des sessions inutiles jusqu'à ce qu'une session soit modifiée).
- **Configuration de Passport.js** : Passport est configuré pour utiliser *LocalStrategy* (stratégie locale). Les fonctions permettant de sérialiser et désérialiser l'utilisateur sont définies pour la gestion des sessions avec Passport.
- **Importation des routes d'authentification** : Un module de routes d'authentification est importé et utilisé avec le préfixe */auth*.
- **Configuration du middleware express.static** : *Express.static* est configuré pour servir les fichiers depuis le dossier *dist*.
- **Démarrage du serveur** : Enfin, le serveur est lancé sur le port spécifié dans la variable d'environnement *process.env.PORT* ou sur le port 3001 s'il n'y a pas de valeur spécifiée.

5.2.4 Présentation de la base de données

J'ai utilisé MongoDB pour la configuration de ma base de données :

EDOUARD'S ORG - 2024-12-03 > PROJECT 0

Database Access

Database Users

Custom Roles

+ ADD NEW DATABASE USER

User ⓘ	Description	Authentication Method ⓘ	MongoDB Roles	Resources	Actions
8801_student_EH		SCRAM	atlasAdmin@admin	All Resources	EDIT DELETE

J'ai ensuite connecté le cluster à mon application en local :

Connect to ClusterMLEXPL



Connecting with MongoDB Driver

1. Select your driver and version

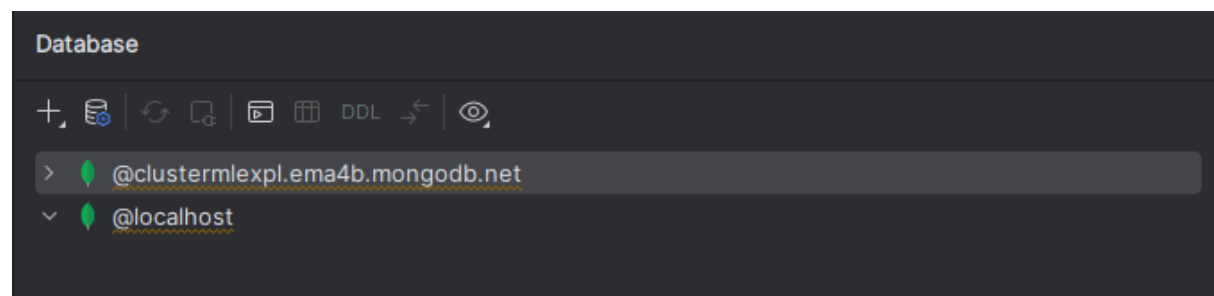
We recommend installing and using the latest driver version.

Driver

Version

Node.js

6.7 or later



Enfin, j'ai pu manager la base de données directement depuis MongoDB Atlas, et ajouter les principales tables de mon application, comme User ou Model :

EDOUARD'S ORG - 2024-12-03 > PROJECT 0 > DATABASES

ClusterMLEXP

VERSION 8.0.3 REGION AWS Paris (eu-west-3)

Overview Real Time Metrics Collections Atlas Search Performance Advisor Online Archive Cmd Line Tools

DATABASES: 2 COLLECTIONS: 7

+ Create Database

Search Namespaces

sample_mflix

comments

embedded_movies

movies

sessions

theaters

users

test

sample_mflix.comments

STORAGE SIZE: 5.98MB LOGICAL DATA SIZE: 11.44MB TOTAL DOCUMENTS: 41079 INDEXES TOTAL SIZE: 1.92MB

Find Indexes Schema Anti-Patterns Aggregation Search Indexes

INSERT DOCUMENT

Filter Type a query: { field: 'value' }

Reset Apply Options

QUERY RESULTS: 1-20 OF MANY

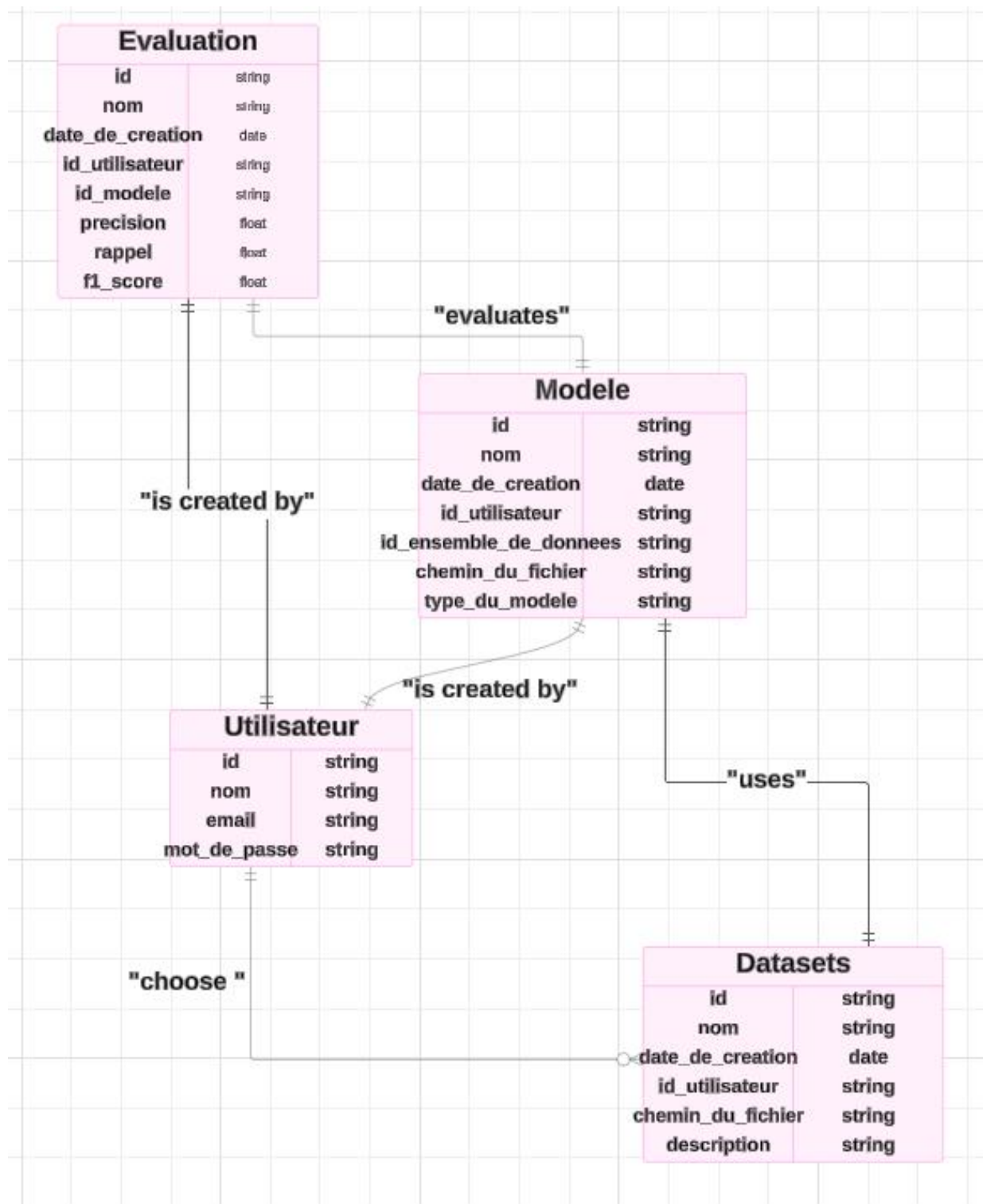
```
{
  "_id": ObjectId("5a9427648b0beeb69579e7"),
  "name": "Mercedes Tyler",
  "email": "mercedes_tyler@fakegmail.com",
  "movie_id": ObjectId("573a1390f29313caabed4323"),
  "text": "Eius veritatis vero facilis quaeat fuga temporibus. Praesentium exped...",
  "date": 2002-08-18T04:56:07.000+00:00
}
```

```
{
  "_id": ObjectId("5a9427648b0beeb69579f5"),
  "name": "John Bishop",
  "email": "john_bishop@fakegmail.com",
  "movie_id": ObjectId("573a1390f29313caabed446f"),
  "text": "Id error ab at molestias dolorum incidunt. Non deserunt praesentium do...",
  "date": 1975-01-21T00:31:22.000+00:00
}
```

Voici un extrait de code qui définit un schéma pour une collection "Users" dans une base de données MongoDB à l'aide de la bibliothèque Mongoose. Chaque document dans la collection "Users" aura un "firstName", un "lastName", un "email" et un "password":

```
JavaScript
1 const mongoose = require("mongoose");
2 const Schema = mongoose.Schema;
3
4 const UserSchema = new Schema({
5   firstName: { type: String, required: true },
6   lastName: { type: String, required: true },
7   email: { type: String, required: true, unique: true },
8   password: { type: String, required: true },
9 });
10
11 module.exports = mongoose.model("User", UserSchema);
```

Voici le diagramme de ma base de données :



Protection contre les injections No-SQL

Avec MongoDB et Mongoose, il y a des mesures de protection intégrées contre les injections NoSQL. Mongoose fait du filtrage et de la validation de données, ce qui aide à prévenir les injections NoSQL. Cependant, il est toujours utile d'ajouter une couche supplémentaire de sécurité.

Valider les entrées : Il est essentiel de valider les entrées côté serveur, même si une validation est également effectuée côté client. Vous pouvez utiliser des bibliothèques comme `joi` or `express-validator` pour valider les entrées.

Utiliser les méthodes Mongoose : Certaines méthodes comme `find()`, `findOne()`, `findOneAndUpdate()`, etc., qui prennent un objet comme argument, sont à l'abri des injections NoSQL.

Éviter \$where et mapReduce: Les fonctions telles que `$where` et `mapReduce` peuvent ouvrir des failles pour les injections NoSQL car elles peuvent contenir du code JavaScript arbitraire. Il est donc conseillé de les éviter.

Protection contre les attaques XSS

Les attaques XSS (Cross-Site Scripting) injectent des scripts dans les entrées qui sont alors sauvegardés et peuvent être exécutés dans le navigateur des autres utilisateurs. Pour s'en prémunir :

Échapper aux entrées : Il est important d'échapper aux entrées utilisateur avant de les afficher. Cela signifie que les caractères tels que `<`, `>`, `{`, `}`, etc. sont convertis en entités HTML sécurisées. Vous pouvez utiliser des bibliothèques comme `he` pour cela.

Utiliser les politiques de sécurité de contenu (CSP) : Les CSP permettent de contrôler les ressources qui sont autorisées à s'exécuter sur votre page, ce qui peut aider à prévenir les attaques XSS.

Configurer les cookies de façon sécurisée : Utilisez les attributs `HttpOnly` et `Secure` pour les cookies. `HttpOnly` empêche l'accès aux cookies via JavaScript. `Secure` garantit que les cookies sont envoyés uniquement via HTTPS.

5.2.5 Création des Routes et Contrôleurs

Au cours du développement de l'application, j'ai défini divers contrôleurs et routes pour gérer les requêtes du client.

Configuration des routes d'authentification : J'ai intégré un module appelé `authRoutes` qui comprenait toutes les routes liées à l'authentification. Ce module a été mis en place dans mon application Express avec le préfixe `/auth`. Cela signifie que toutes les routes définies dans `authRoutes` étaient préfixées par `/auth`.

Création de l'utilisateur (Inscription) : J'ai configuré une route POST `/auth/register` qui a été employée pour l'inscription de nouveaux utilisateurs. L'ensemble des logiques associées à la gestion de cette route a été intégré dans le contrôleur dédié à cette route. (voir **Annexe 12**)

Connexion de l'utilisateur (Login) : J'ai également mis en place une route POST `/auth/login` pour la connexion des utilisateurs existants. La logique nécessaire pour vérifier les identifiants de l'utilisateur et gérer sa session a été définie dans le contrôleur de cette route. (voir **Annexe 13**)

Serveur de fichiers statiques : J'ai configuré mon serveur pour servir des fichiers statiques à partir du répertoire spécifié. Cela signifie que lorsque la requête du client correspond à un fichier existant dans ce répertoire, celui-ci est automatiquement renvoyé au client.

Configuration d'une route catch-all pour servir l'`index.html` : Enfin, une route catch-all est mise en place qui renvoie le fichier `index.html` pour toutes les autres routes qui n'auraient pas été définies précédemment.

Chaque contrôleur est en charge de l'exécution d'une logique spécifique pour répondre à la requête du client. Par exemple, lors de l'inscription d'un nouvel utilisateur, le contrôleur de cette route est responsable de la validation des données envoyées par l'utilisateur, de la création de l'utilisateur dans la base de données, et de l'envoi de la réponse appropriée au client.

5.2.6 Création des middlewares

Au cours du processus de développement, j'ai créé plusieurs middlewares pour gérer différentes tâches dans l'application.

Middleware d'authentification : J'ai mis en place un middleware d'authentification. Ce middleware a été utilisé pour vérifier si un utilisateur était authentifié avant de lui permettre d'accéder à certaines routes. Si l'utilisateur était authentifié, la requête passait au prochain middleware ou au contrôleur de route. Sinon, une réponse avec un code d'erreur approprié était envoyée.

Middleware de gestion des erreurs : J'ai également créé un middleware de gestion des erreurs. Ce middleware a été utilisé pour attraper toutes les erreurs qui étaient lancées dans l'application. Il a généré une réponse avec le bon code d'erreur et le message d'erreur, répondant ainsi à la requête qui avait causé l'erreur.

Middleware de journalisation : J'ai mis en place un middleware de journalisation. Ce middleware a été utilisé pour enregistrer les détails de chaque requête reçue par l'application. Il a facilité le débogage et a permis de garder une trace de l'activité de l'application.

Middleware de traitement des fichiers : J'ai défini un middleware pour le traitement des fichiers. Avec ce middleware, j'ai géré le téléchargement de fichiers par les utilisateurs, les enregistrant dans un répertoire spécifié et ajoutant des informations sur le fichier à la requête pour un traitement ultérieur.

Pour chaque middleware, j'ai utilisé la fonction `app.use()` pour l'ajouter à la pile de middlewares de l'application. Chaque requête entrante devait passer par cette pile de middlewares dans l'ordre dans lequel ils ont été ajoutés.

5.2.7 Extraits de code

Dans cette section, je vais décrire quelques extraits de code essentiels que j'ai écrit pour construire l'API.

Middleware d'authentification : J'ai créé un middleware pour vérifier la présence et la validité d'un JWT dans le header d'authentification d'une requête. Cet extrait de code illustre comment cela a été fait :

```
1 const jwt = require('jsonwebtoken');
2
3 function auth(req, res, next) {
4   const token = req.header('Authorization');
5
6   if (!token) return res.status(401).send('Accès refusé. Token non fourni.');
```

```
7
8   try {
9     const decoded = jwt.verify(token, 'cle_privee');
10    req.user = decoded;
11
12    next();
13  }
14  catch (err) {
15    res.status(400).send('Token invalide.');
```

```
16  }
17 }
18
19 module.exports = auth;
```

Route d'inscription : Pour créer la route d'inscription, j'ai défini une route POST dans mon router Express. Cette route a utilisé un contrôleur pour inscrire un utilisateur :

```
1 const express = require('express');
2 const registerController = require('../controllers/registerController');
3 const router = express.Router();
4
5 router.post('/register', registerController);
6
7 module.exports = router;
```

Contrôleur d'inscription : Le contrôleur d'inscription était responsable de la validation du « username » et « password », de la création d'un nouvel utilisateur et de l'envoi de la réponse approprié :

```
1 const User = require('../models/User');
2
3 function registerController(req, res) {
4   // Validation et création d'un nouvel utilisateur
5   const user = new User(req.body);
6
7   user.save((err, savedUser) => {
8     if (err) return res.status(500).send("Erreur lors de l'inscription de l'utilisateur.");
9
10    return res.status(200).send(savedUser);
11  });
12 }
13
14 module.exports = registerController;
```

5.2.8 Test de l'API

Endpoints / Requests Utilisateur :

- POST api/auth/register : J'ai mis en place cet endpoint pour enregistrer un nouvel utilisateur. Les données d'entrée sont le nom d'utilisateur, l'email et le mot de passe en format JSON. La sortie sera un message de succès ou d'échec, ainsi que les détails de l'utilisateur en cas de succès.
- GET api/auth/login : Cet endpoint est utilisé pour connecter un utilisateur existant. Il accepte l'e-mail et le mot de passe en tant que données d'entrée au format JSON. En cas de succès, il renvoie un token d'authentification.
- PUT api/auth/profile : Cet endpoint permet à l'utilisateur authentifié, de modifier ces informations de profil. Il n'a pas besoin de données d'entrée car il utilise le token d'authentification dans l'en-tête de la requête. Il renvoie les détails de l'utilisateur modifiés
- DELETE /api/users/:id : J'ai conçu cet endpoint pour la suppression des profils utilisateur. Il nécessite l'ID de l'utilisateur comme paramètre dans l'URL. En retour, il émet un message de succès lorsque la suppression est effectuée. À noter que seul un utilisateur authentifié peut initier la suppression de son propre profil. En revanche, la suppression de profils d'autres utilisateurs ne peut être effectuée que par un administrateur.

Pour tester l'API de l'application, j'ai utilisé Postman, qui est un outil formidable qui simplifie le processus de test des API. Voici comment j'ai procédé :

Test de la route d'inscription : J'ai commencé par tester la route d'inscription de l'API (POST /auth/register). Dans Postman, j'ai défini la méthode de requête sur POST, j'ai entré l'URL complète de la route d'inscription, et dans le corps de la requête, j'ai ajouté un objet JSON avec un "username" et un "password" (voir **annexe 14**). Lorsque j'ai envoyé la requête, j'ai reçu en réponse un nouvel objet utilisateur, ce qui m'a indiqué que la route d'inscription fonctionnait correctement.

Test de la route de connexion : Ensuite, j'ai testé la route de connexion (POST /auth/login). De la même manière, j'ai défini la méthode sur POST, j'ai entré l'URL de la connexion, et dans le corps de la requête, j'ai utilisé les mêmes identifiants que j'avais utilisés pour l'inscription. En réponse à cette requête, j'ai reçu un objet JWT (JSON Web Token) qui m'a indiqué que la connexion était un succès.

Test des routes authentifiées : Après avoir réussi à me connecter, j'ai utilisé le JWT pour tester les routes qui nécessitaient une authentification. J'ai ajouté le JWT à l'en-tête 'Authorization' de mes requêtes vers les routes authentifiées. En réponse, j'ai reçu les données attendues, ce qui m'a confirmé que l'authentification fonctionnait correctement.

Test des erreurs : J'ai également testé le comportement de l'API lors de l'envoi de requêtes incorrectes ou malformées. Par exemple, pour la route d'inscription, j'ai essayé d'envoyer une requête sans "username" ni "password", et en réponse, j'ai reçu une erreur appropriée.

L'utilisation de Postman a grandement facilité le processus de test, me permettant de me concentrer sur le développement et l'amélioration de l'API.

5.3 Présentation d'éléments de sécurité de l'application

5.3.1 Utilisation de variable d'environnement

En développant l'API, j'ai constaté qu'il était nécessaire d'utiliser des variables d'environnement pour stocker les informations sensibles, telles que les chaînes de connexion à la base de données et les secrets de l'authentification JWT. Pour ce faire, j'ai utilisé le package dotenv.

Initialisation de dotenv : Pour utiliser dotenv, je l'ai d'abord installé via npm, puis je l'ai importé et initialisé dans mon fichier principal (server.js). Voici comment l'initialisation a été effectuée :

```
1 const dotenv = require('dotenv');
2 dotenv.config();
```

Définition des variables dans .env : J'ai créé un fichier .env dans la racine du projet et défini mes variables d'environnement dans ce fichier. Par exemple, j'ai défini la chaîne de connexion à la base de données :

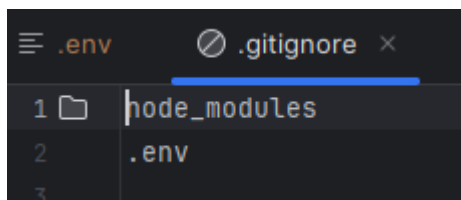
```
1 DB_CONNECT=mongodb://your-database-url
2 JWT_SECRET=your-random-secret
```

{exemple d'une variable de connexion à mongoDB et d'un token JWT}

Utilisation des variables d'environnement : Ensuite, j'ai utilisé ces variables dans mon code en les appelant via l'objet process.env. Par exemple :

```
1 mongoose.connect(process.env.DB_CONNECT, { useNewUrlParser: true });
2 ...
3 const token = jwt.sign({...payload}, process.env.JWT_SECRET);
```

Sécurité : Il est crucial de ne pas commettre le fichier .env dans le dépôt git pour empêcher la fuite d'informations sensibles. Pour ce faire, j'ai ajouté .env dans mon fichier .gitignore. :



L'utilisation de dotenv et des variables d'environnement a permis de maintenir les informations sensibles en dehors du code source, ce qui renforce la sécurité et la flexibilité du projet.

5.3.2 Configuration CORS

La configuration de CORS (Cross-Origin Resource Sharing) est une pratique essentielle pour contrôler comment les ressources de notre serveur peuvent être accessibles à partir de différents domaines. Cela est d'autant plus important lorsqu'il s'agit d'un serveur d'API REST, où les requêtes peuvent venir de différentes origines.

J'utilise le module cors de npm pour configurer CORS dans mon application Express. Voici le processus que j'ai suivi :

Installation du module CORS : J'ai commencé par installer le middleware cors via npm comme suit : `npm install cors`.

Importation et utilisation du middleware : J'ai ensuite importé le middleware cors dans mon fichier `server.js` et je l'ai ajouté à l'application Express comme suit :

```
1 const express = require('express');
2 const cors = require('cors');
3 const app = express();
4 app.use(cors());
```

5.3.3 Sécurités lors de la création d'un compte utilisateur

Lors de la création de comptes utilisateurs, j'ai mis en place plusieurs mesures pour garantir la sécurité des données de l'utilisateur.

Hashage du mot de passe : Lors de la création du compte, j'ai veillé à ce que le mot de passe fourni par l'utilisateur soit hashé avant d'être stocké en base de données. Pour cela, j'ai utilisé la bibliothèque `bcryptjs`.

Voici un exemple de la manière dont j'ai procédé :

```
1 const bcrypt = require('bcryptjs');
2
3 // ...
4
5 async function registerController(req, res) {
6   // ... validation des données
7
8   // Hashage du password
9   const salt = await bcrypt.genSalt(10);
10  const hashedPassword = await bcrypt.hash(req.body.password, salt);
11
12  // Création d'un nouvel utilisateur
13  const user = new User({
14    username: req.body.username,
15    password: hashedPassword
16  });
17
18  // ... enregistrement de l'utilisateur dans la base de données
19 }
```

5.3.4 Authentification via Json Web Token (JWT)

Dans mon projet ML Explorer, j'ai mis en place le processus d'authentification via Json Web Token (JWT). Voici je l'ai fait :

Création d'un token JWT : Lorsqu'un utilisateur s'authentifie avec succès (après avoir vérifié son nom d'utilisateur et son mot de passe), j'ai créé un token JWT. Pour cela, j'ai utilisé le package jsonwebtoken. Voici un exemple :

```
1 const jwt = require('jsonwebtoken');
2
3 function login(req, res) {
4   const user = // obtenir l'utilisateur de la base de données...
5
6   const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
7     expiresIn: process.env.JWT_EXPIRES_IN,
8   });
9
10  res.status(200).json({
11    status: 'success',
12    token
13  });
14 }
```

Dans cet exemple, `process.env.JWT_SECRET` est le secret utilisé pour signer le token et `process.env.JWT_EXPIRES_IN` est la durée de validité du token.

Envoi du token à l'utilisateur : Une fois le token créé, je l'ai envoyé à l'utilisateur. Il sera stocké côté client et envoyé avec chaque requête nécessitant une authentification.

Vérification du token : Pour les routes nécessitant une authentification, un middleware vérifie si le token envoyé dans les headers de la requête. Si le token est valide, l'utilisateur est authentifié et la requête peut continuer. Si le token n'est pas valide (par exemple, s'il est expiré), une erreur est renvoyée.

5.4 Présentation des tests

J'ai porté une attention particulière à tester les éléments de sécurité. J'ai utilisé pour cela jest, un cadre de tests populaire pour JavaScript.

Tests d'authentification : J'ai écrit des tests pour vérifier le bon fonctionnement de l'authentification des utilisateurs. Cela comprenait des tests pour s'assurer qu'un token valide était bien délivré lors de l'enregistrement d'un nouvel utilisateur et lors de la connexion d'un utilisateur existant.

Tests des routes sécurisées : J'ai vérifié que seuls les utilisateurs authentifiés pouvaient accéder aux routes sécurisées. J'ai fait cela en envoyant des requêtes à ces routes sans token JWT, et j'ai vérifié que ces requêtes échouaient.

Tests de hashage du mot de passe : J'ai écrit des tests pour confirmer que les mots de passe étaient correctement hashés avant d'être stockés dans la base de données.

En somme, tester les éléments de sécurité a été un aspect essentiel de mon travail pour m'assurer que les informations des utilisateurs étaient bien protégées. Cette série de tests a permis de garantir que le système d'authentification fonctionne correctement et respecte les standards de sécurité modernes.

Voici un tableau qui récapitule l'ensemble de ces tests :

s scénario de test	r résultat attendu	r résultat retourné
<u>user-authentication.test.js:</u>		
Vérifie si la fonction d'authentification d'un utilisateur fonctionne correctement lorsque les identifiants valides sont fournis. Il utilise la fonction authenticateUser	Un utilisateur soit retourné lorsque les identifiants valides sont transmis, et que le nom d'utilisateur retourné corresponde au nom d'utilisateur passé à la fonction "authenticateUser"	"authentifie un utilisateur avec des identifiants valides"
<u>admin-routes.test.js:</u>		
Vérifie si une route sécurisée dans une application Express nécessite une authentification	recevoir un statut de réponse 401, ce qui indique qu'une authentification est nécessaire	"la route nécessite une authentification"
<u>admin-password-hash.test.js:</u>		
Vérifie si la fonction de hachage du mot de passe fonctionne correctement utilise la fonction hashPassword	la fonction `hashPassword` et s'attend à ce que le mot de passe haché retourné soit différent du mot de passe en texte brut.	"hache le mot de passe correctement"

5.5 Description de la veille, vulnérabilités de sécurité

Dans le cadre de mon projet, la surveillance constante des problèmes et des vulnérabilités de sécurité a été une partie importante de mes responsabilités. J'ai donc procédé ainsi :

J'ai commencé par m'abonner aux bulletins de sécurité pour les technologies que j'ai utilisées dans mon projet, notamment Node.js, Express et les packages NPM tels que jsonwebtoken, bcryptjs. Ces bulletins ont souvent été ma première source d'information sur les nouvelles vulnérabilités détectées et les correctifs disponibles.

En outre, j'ai utilisé des outils d'analyse de sécurité statique sur mon code pour détecter les vulnérabilités potentielles. J'ai notamment utilisé eslint-plugin-security, un plugin ESLint qui analyse le code à la recherche de motifs connus pour présenter des risques de sécurité.

J'ai également effectué des audits de sécurité réguliers sur mes dépendances NPM à l'aide de la commande npm audit. Cette commande vérifie les dépendances par rapport à une base de données de vulnérabilités connues et propose des corrections lorsque c'est possible.

De plus, j'ai cherché activement à comprendre les meilleures pratiques de sécurité et je me suis efforcé de les implémenter dans mon projet. Par exemple, j'ai pris soin de toujours hasher les mots de passe avant de les stocker et de valider et d'assainir les entrées utilisateur pour prévenir les attaques d'injection.

Enfin, j'ai fait un effort pour rester au courant des dernières nouvelles et des dernières recherches en matière de sécurité, en lisant régulièrement des blogs et des articles sur le sujet. J'ai trouvé cela essentiel pour comprendre l'évolution constante du paysage de la sécurité et pour m'assurer que mon application reste aussi sécurisée que possible face aux menaces émergentes.

En résumé, la veille en matière de sécurité a été une activité continue tout au long de mon projet. Elle m'a aidé à anticiper et à réagir rapidement aux vulnérabilités, en assurant la sécurité de mon application et la protection des données de mes utilisateurs.

6 Conclusion

Pour conclure, la réalisation de ce projet a été une expérience incroyablement enrichissante pour moi. J'ai pu développer mes compétences en programmation JavaScript, tout en me familiarisant avec des outils et des cadres de travail tels que Node.js, Express, et React. J'ai eu la possibilité d'approfondir ma compréhension des

systèmes de gestion de bases de données avec MongoDB et Mongoose, et j'ai également acquis une expérience précieuse dans la manipulation des API REST.

La création d'un système d'authentification sûr à l'aide de JWT m'a permis de comprendre en détail comment fonctionne l'authentification et la sécurité dans les applications modernes. L'écriture de tests solides pour l'application était un défi intéressant, et j'ai rapidement réalisé la valeur de ces tests pour maintenir la qualité du code tout en ajoutant de nouvelles fonctionnalités.

Finalement, la surveillance continue des problèmes et des vulnérabilités de sécurité m'a donné une bonne idée de l'importance de la sécurité dans le développement d'applications et du rôle essentiel que joue la veille en matière de sécurité pour rester au courant des menaces émergentes.

Dans l'ensemble, j'estime que ce projet a considérablement renforcé mes compétences en développement full stack et en sécurité de l'information. Je suis impatient d'appliquer ces compétences acquises à de futurs projets.

7 Annexes

Annexe 1 :

DataTable.jsx

```
DataTable.jsx x
1  /* React component to handle displaying imported data. */
2  import PropTypes from "prop-types";
3  import React, { Component } from "react";
4  import { connect } from "react-redux";
5  import { getTableData, setCurrentColumn, setHighlightColumn } from "../redux";
6  import { styles } from "../constants";
7  import { getLocalizedColumnName } from "../helpers/columnDetails.js";
8
9  class DataTable extends Component { Show usages  Edouard26
10     static propTypes : {...} = {
11         currentPanel: PropTypes.string,
12         data: PropTypes.array,
13         datasetId: PropTypes.string,
14         labelColumn: PropTypes.string,
15         selectedFeatures: PropTypes.array,
16         setCurrentColumn: PropTypes.func,
17         setHighlightColumn: PropTypes.func,
18         currentColumn: PropTypes.string,
19         highlightColumn: PropTypes.string,
20         reducedColumns: PropTypes.bool,
21         singleRow: PropTypes.number,
22         startingRow: PropTypes.number,
23         noLabel: PropTypes.bool,
24         hideLabel: PropTypes.bool
25     };
26
27     getColumnHeaderStyle = key => { Show usages  Edouard26
28         let style;
29
30         if (key === this.props.labelColumn) {
31             style = styles.dataDisplayHeaderLabel;
32         } else if (this.props.selectedFeatures.includes(key)) {
33             style = styles.dataDisplayHeaderFeature;
34         }
35
36         const pointerStyle : false | {...} =
37             !this.props.reducedColumns && styles.dataDisplayHeaderClickable;
38
39         return { ...styles.dataDisplayHeader, ...pointerStyle, ...style };
40     };
41 }
```

```
41
42 getColumnCellStyle = key => { Show usages  Edouard26
43   let style;
44
45   if (this.props.hideLabel && this.props.labelColumn === key) {
46     style = styles.dataDisplayCellHidden;
47   } else if (key === this.props.currentColumn) {
48     if (this.props.currentPanel === "dataDisplayLabel") {
49       style = styles.dataDisplayCellSelectedLabel;
50     } else {
51       style = styles.dataDisplayCellSelected;
52     }
53   } else if (key === this.props.highlightColumn) {
54     if (this.props.currentPanel === "dataDisplayLabel") {
55       style = styles.dataDisplayCellHighlightedLabel;
56     } else {
57       style = styles.dataDisplayCellHighlighted;
58     }
59   }
60
61   return { ...styles.dataDisplayCell, ...style };
62 };
63
64 getColumnns = () :string[] => { Show usages  Edouard26
65   if (this.props.reducedColumns) {
66     return Object.keys(this.props.data[0])
67       .filter(key :string => {
68         return (
69           (!this.props.noLabel && this.props.labelColumn === key) ||
70           this.props.selectedFeatures.includes(key)
71         );
72       })
73       .sort( compareFn: (key1 :string , key2 :string ) :number => {
74         return this.props.labelColumn === key1 ? 1 : -1;
75       });
76   }
77
78   return Object.keys(this.props.data[0]);
79 };
```

```
return (
  <table style={styles.displayTable}>
    <thead>
      <tr>
        {this.getColumns().map(columnId :string => {
          return (
            <th
              key={columnId}
              style={this.getColumnHeaderStyle(columnId)}
              onClick={() => this.setCurrentColumn(columnId)}
              onKeyDown={() => this.setCurrentColumn(columnId)}
              onMouseEnter={() => this.setHighlightColumn(columnId)}
              onMouseLeave={() => this.setHighlightColumn(columnId: undefined)}
            >
              {getLocalizedColumnName(this.props.datasetId, columnId)}
            </th>
          );
        })}
      </tr>
    </thead>
    <tbody>
      {this.getRows().map((row :T, index :number) => {
        return (
          <tr key={index}>
            {this.getColumns().map(columnId :string => {
              return (
                <td
                  key={columnId}
                  className="uitest-data-table-column"
                  style={this.getColumnCellStyle(columnId)}
                  onClick={() => this.setCurrentColumn(columnId)}
                  onKeyDown={() => this.setCurrentColumn(columnId)}
                  onMouseEnter={() => this.setHighlightColumn(columnId)}
                  onMouseLeave={() => this.setHighlightColumn(columnId: undefined)}
                  role="presentation"
                >
                  {startingRow !== undefined && index <= startingRow ? (
                    <span>&nbsp;&nbsp;&nbsp;</span>

```

Annexe 2:

DataDisplay.jsx

```
DataDisplay.jsx x
1  /* React component to handle displaying imported data. */
2  > import ...
9
10 class DataDisplay extends Component { Show usages  Edouard26
11   static propTypes : {data: any} = {
12     data: PropTypes.array
13   };
14
15   render() { no usages  Edouard26
16     const { data } = this.props;
17
18     if (data.length === 0) {
19       return null;
20     }
21
22     const rowCount : number = this.props.data.length;
23     const rowLimit : number = 100;
24     const rowCountMessage : string = (rowCount <= rowLimit) ?
25       I18n.t( key: "dataDisplayRowCount", options: {"rowCount": rowCount} ) :
26       I18n.t( key: "dataDisplayRowCountTruncated", options: {"rowCount": rowCount, "rowLimit": rowLimit});
27     return (
28       <div id="data-display" style={styles.panel}>
29         <Statement />
30         <div style={styles.tableParent}>
31           <DataTable />
32         </div>
33         <div style={styles.footerText}>
34           {rowCountMessage}
35         </div>
36       </div>
37     );
38   }
39 }
40
41 export default connect( Show usages  Edouard26
42   mapStateToProps: state => ({
43     data: state.data
44   })
45 )(DataDisplay);
```

Annexe3:

DataCard.jsx

```

1  /* React component to show information about the currently-selected data set. */
2  > import ...
10
11  class DataCard extends Component { Show usages  Edouard26
12    static propTypes : {...} = {
13      name: PropTypes.string,
14      metadata: metadataShape,
15      datasetDetails: datasetDetailsShape,
16      dataLength: PropTypes.number,
17      removedRowCount: PropTypes.number
18    };
19
20  @render() { no usages  Edouard26
21
22    const { name, metadata, datasetDetails, dataLength, removedRowCount } = this.props;
23
24    const card = metadata && metadata.card;
25
26    const dataLengthLimit :number = 20000;
27    if (dataLength > dataLengthLimit) {
28      window.alert(
29        I18n.t( key: "dataCardWarningLargeDataset", options: {"rowCount": dataLengthLimit})
30      );
31    }
32
33    const removedRowsMsg :string| null =
34      removedRowCount > 0
35      ? I18n.t( key: "dataCardRemovedRows", options: {"rowCount": removedRowCount})
36      : null;
37

```

```

38  return (
39    dataLength !== 0 && (
40      <div id="data-card" style={...styles.panel, ...styles.rightPanel }>
41        <div style={styles.largeText}>{name || I18n.t( key: "dataCardDefaultHeader")}</div>
42        <ScrollableContent>
43          {card && (
44            <div>
45              <div style={styles.cardRow}>{datasetDetails.description}</div>
46              <div style={styles.cardRow}>
47                <span style={styles.italic}>
48                  {I18n.t( key: "dataCardSource")}
49                <br />
50                {metadata.card.source}
51              </span>
52            </div>
53            <div style={styles.cardRow}>
54              <span style={styles.italic}>
55                {I18n.t( key: "dataCardRowCount")}
56              <br />
57              {dataLength}
58            </span>
59            </div>
60            {metadata.card.lastUpdated && (
61              <div style={styles.cardRow}>
62                <span style={styles.italic}>
63                  {I18n.t( key: "dataCardLastUpdated")}
64                <br />
65                {metadata.card.lastUpdated}
66              </span>
67            </div>
68          )}
69
70            {metadata.card.context.potentialUses && (
71              <div style={styles.cardRow}>
72                <div style={styles.bold}>{I18n.t( key: "dataCardPotentialUses")}</div>
73                <div style={styles.italic}>
74                  {datasetDetails.potentialUses}
75                </div>
76              </div>

```

Annexe 4:

Model Card.jsx

```

ModelCard.jsx x
1  /* React component to handle displaying the model card. */
2  import PropTypes from "prop-types";
3  import React, { Component } from "react";
4  import { connect } from "react-redux";
5  import { styles } from "../constants";
6  import { getLabelToSave, getFeaturesToSave } from "../redux";
7  import { getPercentCorrect } from "../helpers/accuracy";
8  import { getDatasetDetails } from "../helpers/datasetDetails";
9  import Statement from "../Statement";
10 import aiBotBorder from "../images/ai-bot/ai-bot-border.png";
11 import { datasetDetailsShape, trainedModelDetailsShape, modelCardColumnShape } from "../shape";
12 import I18n from "../i18n";
13 import { getLocalizedColumnName } from "../helpers/columnDetails.js";
14
15 class ModelCard extends Component { Show usages  Edouard26
16   static propTypes: {...} = {
17     trainedModelDetails: trainedModelDetailsShape,
18     selectedFeatures: PropTypes.arrayOf(modelCardColumnShape),
19     percentCorrect: PropTypes.string,
20     label: modelCardColumnShape,
21     features: PropTypes.arrayOf(PropTypes.string),
22     datasetDetails: datasetDetailsShape
23   };
24
25   render() { no usages  Edouard26
26     const {
27       trainedModelDetails,
28       selectedFeatures,
29       percentCorrect,
30       label,
31       features,
32       datasetDetails
33     } = this.props;
34     console.log("trainedModelDetails", trainedModelDetails)
35     const localizedLabel: string = getLocalizedColumnName(datasetDetails.name, label.id);
36     const localizedFeatures =
37       selectedFeatures.map(feature => getLocalizedColumnName(datasetDetails.name, feature));
38     const predictionStatement: string = I18n.t( key: "predictionStatement",
39       options: {"output": localizedLabel, "inputs": localizedFeatures.join(", ")}
40     );
41     return (
42       <div style={styles.panel}>
43         <Statement />

```

```
40     return (  
41         <div style={styles.panel}>  
42             <Statement />  
43             <div style={styles.summaryScreenBot}>  
44                 <img  
45                     src={aiBotBorder}  
46                     className="ailab-image-hover"  
47                     style={styles.summaryScreenBotImage}  
48                     alt={I18n.t( key: "aiBotAltText")}  
49                 />  
50             </div>  
51             <div id="vittest-model-card" style={styles.modelCardContainer}>  
52                 <h3 style={styles.modelCardHeader}>{trainedModelDetails.name}</h3>  
53                 <div style={styles.modelCardSubpanel}>  
54                     <h5 style={styles.modelCardHeading}>{I18n.t( key: "modelCardAccuracy")}</h5>  
55                     <div style={styles.modelCardContent}>  
56                         <p style={styles.modelCardDetails}>  
57                             <span>  
58                                 {percentCorrect}%  
59                             </span>  
60                         </p>  
61                     </div>  
62                 </div>  
63                 <div style={styles.modelCardSubpanel}>  
64                     <h5 style={styles.modelCardHeading}>{I18n.t( key: "modelCardIntendedUse")}</h5>  
65                     <div style={styles.modelCardContent}>  
66                         {trainedModelDetails.potentialUses && (  
67                             <p style={styles.modelCardDetails}>  
68                                 {trainedModelDetails.potentialUses}  
69                             </p>  
70                         )}  
71                     </div>  
72                 </div>  
73                 <div style={styles.modelCardSubpanel}>  
74                     <h5 style={styles.modelCardHeading}>{I18n.t( key: "modelCardLimitations")}</h5>  
75                     <div style={styles.modelCardContent}>  
76                         {trainedModelDetails.potentialMisuses && (  
77                             <p style={styles.modelCardDetails}>  
78                                 {trainedModelDetails.potentialMisuses}
```

Annexe 5:

Predict.jsx

```

1  /* React component to handle predicting and displaying predictions. */
2  > import { useState, useEffect } from 'react';
20
21  class Predict extends Component { Show usages  ⚡ Edouard26
22    static propTypes: {...} = {
23      labelColumn: PropTypes.string,
24      selectedCategoricalFeatures: PropTypes.array,
25      selectedNumericalFeatures: PropTypes.array,
26      uniqueOptionsByColumn: PropTypes.object,
27      testData: PropTypes.object,
28      setTestData: PropTypes.func.isRequired,
29      predictedLabel: PropTypes.oneOfType([PropTypes.string, PropTypes.number]),
30      getPredictAvailable: PropTypes.bool,
31      extremaByColumn: PropTypes.object,
32      datasetId: PropTypes.string
33    };
34
35    handleChange = (event, feature) : void => { Show usages  ⚡ Edouard26
36      this.props.setTestData(feature, event.target.value);
37    };
38
39    onClickPredict = () : void => { Show usages  ⚡ Edouard26
40      train.onClickPredict(store);
41    };
42
43    render() { no usages  ⚡ Edouard26
44      const { datasetId, labelColumn, predictedLabel } = this.props;
45      return (
46        <div id="predict" style={{ ...styles.panel, ...styles.rightPanel }}>
47          <div style={styles.largeText}>{I18n.t( key: "predictHeader")}</div>
48          <ScrollableContent>
49            <div>
50              {this.props.selectedNumericalFeatures.map((feature, index : number) => {
51                let min = this.props.extremaByColumn[feature].min.toFixed(2);
52                let max = this.props.extremaByColumn[feature].max.toFixed(2);
53
54                return (
55                  <div style={styles.cardRow} key={index}>
56                    <label>
57                      {getLocalizedColumnName(datasetId, feature)}
58                      &nbsp;
59                    <input

```

Annexe 6:

ResulDetails.jsx

```
ResultsDetails.jsx x
1  /* React component to handle displaying accuracy results. */
2  > import ...
17
18  class ResultsDetails extends Component { Show usages  ⚡ Edouard26
19    static propTypes : {...} = {
20      resultsTab: PropTypes.string,
21      selectedFeatures: PropTypes.array,
22      labelColumn: PropTypes.string,
23      percentCorrect: PropTypes.string,
24      setShowResultsDetails: PropTypes.func,
25      correctResults: resultsPropType,
26      incorrectResults: resultsPropType,
27    };
28
29    onClose = () :void => { Show usages  ⚡ Edouard26
30      this.props.setShowResultsDetails(false);
31    };
32
33    @render() { no usages  ⚡ Edouard26
34      const results =
35        this.props.resultsTab === ResultsGrades.CORRECT
36          ? this.props.correctResults
37          : this.props.incorrectResults;
38
39      return (
40        <div style={styles.panelPopupContainer}>
41          <div id="results-details" style={styles.panelPopup}>
42            <div
43              onClick={this.onClose}
44              onKeyDown={this.onClose}
45              style={styles.popupClose}
46              role="button"
47              tabIndex={0}
48            >
49              <FontAwesomeIcon icon={faTimes} />
50            </div>
51            {!isNaN(this.props.percentCorrect) && <ResultsToggle />}
52            <ResultsTable results={results} />
53          </div>
54        </div>
55      );
56    }
```

Annexe 7 :

ResultTable.jsx

```
ResultTable.jsx x
1  /* React component to handle displaying test data and A.I. Bot's guesses. */
2  > import ...
10
11  class ResultsTable extends Component { Show usages  Edouard26
12    static propTypes: {...} = {
13      selectedFeatures: PropTypes.array,
14      labelColumn: PropTypes.string,
15      results: resultsPropType,
16      isRegression: PropTypes.bool,
17      setResultsHighlightRow: PropTypes.func,
18      resultsHighlightRow: PropTypes.number,
19      datasetId: PropTypes.string
20    };
21
22    getRowCellStyle = index => { Show usages  Edouard26
23      return {
24        ...styles.tableCell,
25        ...(index === this.props.resultsHighlightRow &&
26          styles.resultsCellHighlight)
27      };
28    };
29
30    render() { no usages  Edouard26
31      const { setResultsHighlightRow, datasetId } = this.props;
32      const featureCount: number = this.props.selectedFeatures.length;
33
34      return (
35        <div style={styles.panel}>
36          {this.props.isRegression && (
37            <div style={styles.smallTextRight}>
38              {I18n.t(
39                key: "resultsTablePredictionRange",
40                options: {"percentage": REGRESSION_ERROR_TOLERANCE}
41              )}
42            </div>
43          )}
44        </div>
```

```

11  class ResultsTable extends Component { Show usages  Edouard26
30  render() { no usages  Edouard26
34      return (
35          <div style={styles.panel}>
36              {this.props.isRegression && (
37                  <div style={styles.smallTextRight}>
38                      {I18n.t(
39                          key: "resultsTablePredictionRange",
40                          options: {"percentage": REGRESSION_ERROR_TOLERANCE}
41                      )}
42                  </div>
43              )}
44
45          <div style={styles.tableParent}>
46              <table style={styles.displayTable}>
47                  <thead>
48                      <tr>
49                          <th
50                              colSpan={featureCount}
51                              style={{
52                                  ...styles.tableHeader,
53                                  ...styles.resultsTableFirstHeader
54                              }}
55                          >
56                              {I18n.t( key: "resultsTableFeatureHeader")}
57                          </th>
58                          <th
59                              style={{
60                                  ...styles.tableHeader,
61                                  ...styles.resultsTableFirstHeader
62                              }}
63                          >
64                              {I18n.t( key: "resultsTableActualValueHeader")}
65                          </th>
66                          <th
67                              style={{
68                                  ...styles.tableHeader,
69                                  ...styles.resultsTableFirstHeader
70                              }}
71                          >

```

Annexe 8 :

SaveModel.jsx

```
SaveModel.jsx x
1  /* React component to handle saving a trained model. */
2  > import ...
16
17  class SaveModel extends Component { Show usages  ⚡ Edouard26
18    static propTypes : {...} = {
19      trainedModel: PropTypes.object,
20      setTrainedModelDetail: PropTypes.func,
21      trainedModelDetails: PropTypes.object,
22      labelColumn: PropTypes.string,
23      columnDescriptions: PropTypes.array,
24      dataDescription: PropTypes.string,
25      isUserUploadedDataset: PropTypes.bool,
26      datasetId: PropTypes.string
27    };
28
29    constructor(props) { no usages  ⚡ Edouard26
30      super(props);
31
32      this.state = {
33        showColumnDescriptions: this.props.isUserUploadedDataset
34      };
35    }
36
37    toggleColumnDescriptions = () :void => { Show usages  ⚡ Edouard26
38      this.setState( state: {
39        showColumnDescriptions: !this.state.showColumnDescriptions
40      });
41    };
42
43    handleChange = (event, field, isColumn) :void => { Show usages  ⚡ Edouard26
44      this.props.setTrainedModelDetail(field, event.target.value, isColumn);
45    };
46
47    getColumnFields = () :[] => { Show usages  ⚡ Edouard26
48      var fields :any[] = [];
49
50      for (const columnDescription of this.props.columnDescriptions) {
51        const datasetId :string = this.props.datasetId;
52        const labelType :string = I18n.t( key: "saveModelColumnTypeLabel");
53        const featureType :string = I18n.t( key: "saveModelColumnTypeFeature");
54        const columnType :string =
55          columnDescription.id === this.props.labelColumn ? labelType : featureType
```

```

108
109     return (
110       <div id="vittest-model-card-form" style={styles.panel}>
111         <Statement />
112         <ScrollableContent tinted={true}>
113           <div key={nameField.id} style={styles.cardRow}>
114             <span style={styles.bold}>{nameField.text}</span>
115             &nbsp;
116             <span style={styles.italic}>{I18n.t( key: "saveModelFieldRequired")}</span>
117             <div>
118               <input
119                 id="vittest-model-name-input"
120                 onChange={event : ChangeEvent<HTMLInputElement> =>
121                   this.handleChange(event, nameField.id, nameField.isColumn)
122                 }
123                 maxLength={ModelNameMaxLength}
124               />
125             </div>
126           </div>
127           <div>
128             {this.getUsesFields().map(field => {
129               return (
130                 <div key={field.id} style={styles.cardRow}>
131                   <div style={styles.bold}>{field.text}</div>
132                   <div>{field.description}</div>
133                   <ul>
134                     {field.descriptionDetails &&
135                       field.descriptionDetails.map((detail : T , index : number ) => {
136                         return (
137                           <li style={styles.regularText} key={index}>
138                             {detail}
139                           </li>
140                         );
141                       })}
142                     </ul>
143                     {!field.answer && (
144                       <div>
145                         <textarea

```

Annexe 9:

SelectDataset.jsx

```
SelectDataset.jsx x
1  /* React component to handle importing CSVs and pushing data to Redux store. */
2  > import ...
19
20  class SelectDataset extends Component { Show usages  ⚡ Edouard26 +1
21    static propTypes: {...} = {
22      setSelectedName: PropTypes.func.isRequired,
23      setSelectedCSV: PropTypes.func.isRequired,
24      setSelectedJSON: PropTypes.func.isRequired,
25      setHighlightDataset: PropTypes.func.isRequired,
26      csvfile: PropTypes.oneOfType([PropTypes.string, PropTypes.object]),
27      jsonfile: PropTypes.oneOfType([PropTypes.string, PropTypes.object]),
28      resetState: PropTypes.func.isRequired,
29      specifiedDatasets: PropTypes.arrayOf(PropTypes.string),
30      name: PropTypes.string,
31      highlightDataset: PropTypes.string,
32      invalidData: PropTypes.string
33    };
34
35    constructor(props) { no usages  ⚡ Edouard26
36      super(props);
37
38      this.state = {
39        download: false
40      };
41    }
42
43    handleDatasetClick = id => { Show usages  ⚡ Edouard26 +1
44      const assetPath = global.__ml_explorer_asset_public_path__;
45      const dataset: Object = getDatasets().find(dataset: Object => dataset.id === id);
46
47      // Don't process the click if we're just clicking the current
48      // dataset again.
49      if (dataset.name !== this.props.name) {
50        const csvPath = assetPath + dataset.path;
51        const jsonPath = assetPath + dataset.metadataPath;
52
53        this.props.resetState();
54        this.props.setSelectedName(dataset.name);
55        this.props.setSelectedCSV(csvPath);
56        this.props.setSelectedJSON(jsonPath);
57        this.setState({ state: {
58          download: true
```

```

95
96     return (
97       <div id="select-dataset" style={styles.panel}>
98         <ScrollableContent tinted={true}>
99           {datasets.map((dataset :Object , index :number ) => {
100             return (
101               <div
102                 style={{
103                   ...styles.selectDatasetItem,
104                   ...(this.props.highlightDataset === dataset.name &&
105                     styles.selectDatasetItemHighlighted),
106                   ...(this.props.name === dataset.name &&
107                     styles.selectDatasetItemSelected),
108                   ...(index % 3 === 0 && { clear: "both" })
109                 }}
110                 key={dataset.id}
111                 onClick={() => this.handleDatasetClick(dataset.id)}
112                 onKeyDown={() => this.handleDatasetClick(dataset.id)}
113                 onMouseEnter={() =>
114                   this.props.setHighlightDataset(dataset.name)
115                 }
116                 onMouseLeave={() => this.props.setHighlightDataset(undefined)}
117                 role="button"
118                 tabIndex={0}
119               >
120                 <div style={styles.selectDatasetItemContainer}>
121                   <img
122                     src={assetPath + dataset.imagePath}
123                     style={styles.selectDatasetImage}
124                     draggable={false}
125                     className="uittest-ailab-dataset-image ailab-image-hover"
126                     alt={`Select ${dataset.name} dataset`}
127                   />
128                   <div style={styles.selectDatasetText}>{dataset.name}</div>
129                 </div>
130               </div>
131             );
132           })}
133         </ScrollableContent>

```

Annexe 10:

TrainModel.jsx

```

1  |/* React component to handle training. */
2  |> import { useState, useEffect } from 'react';
15
16  |const framesPerCycle :number = 80;
17  |const maxNumItems :number = 7;
18
19  |class TrainModel extends Component { Show usages  ⚡ Edouard26
20  |  static propTypes :{...} = {
21  |    data: PropTypes.array,
22  |    readyToTrain: PropTypes.bool,
23  |    labelColumn: PropTypes.string,
24  |    selectedFeatures: PropTypes.array,
25  |    instructionsOverlayActive: PropTypes.bool
26  |  };
27
28  |  constructor(props) { no usages  ⚡ Edouard26
29  |    super(props);
30
31  |    this.state = {
32  |      frame: 0,
33  |      animationTimer: undefined,
34  |      headOpen: false,
35  |      finished: false
36  |    };
37  |  }
38
39  |  componentDidMount() :void { no usages  ⚡ Edouard26
40  |    train.init(store);
41  |    train.onClickTrain(store);
42
43  |    const animationTimer :number = setInterval(
44  |      this.updateAnimation.bind(this),
45  |      timeout: 1000 / 30
46  |    );
47
48  |    this.setState( state: { animationTimer });
49  |  }
50
51  |  updateAnimation = () :void => { Show usages  ⚡ Edouard26
52  |    if (this.state.frame === 15) {
53  |      this.setState( state: { headOpen: true });
54  |    }

```

```
116     return (  
117       <div  
118         id="train-model"  
119         style={{  
120           ...styles.panel,  
121           justifyContent: "center",  
122           backgroundSize: "cover",  
123           backgroundPosition: "50% 50%",  
124           backgroundImage: "url(" + background + ")"  
125         }}  
126       >  
127         <div style={styles.statementWithBackgroundAbsolute}>{I18n.t(key: "trainModelHeading")}</div>  
128  
129         <div  
130           style={{  
131             ...styles.trainModelDataTable,  
132             opacity: tableOpacity  
133           }}  
134         >  
135           <DataTable reducedColumns={true} startingRow={startingRow} />  
136         </div>  
137  
138         <div style={styles.trainModelContainer}>  
139           {showAnimation && (  
140             <div  
141               style={{  
142                 position: "absolute",  
143                 transformOrigin: "center center",  
144                 top: translateY + "%",  
145                 left: translateX + "%",  
146                 transform: transform,  
147                 opacity: opacity * tableOpacity  
148               }}  
149             >  
150               <DataTable  
151                 reducedColumns={true}  
152                 singleRow={this.getAnimationStep()}  
153               />  
154             </div>  
155           )  
156         </div>  
157       )  
158     )  
159   )  
160 }
```

Annexe 11:**Server.js**

```
server.js x
1 // server.js
2 require('dotenv').config();
3 const express : e | () => core.Express = require('express');
4 const path : PlatformPath | path = require('path');
5 const mongoose : ... = require('mongoose');
6 const passport : Authenticator | {...} = require('passport');
7 const LocalStrategy : function(Object, Function): vo... | {...} = require('passport-local').Strategy;
8 const session : function(cookie?: Object, gen... | {...} = require('express-session');
9 const bcrypt : any | {...} = require('bcryptjs');
10 const flash : function(any): function(any, a... | {...} = require('connect-flash');
11 const app : any | Express = express();
12 const jwt : {...} = require('jsonwebtoken');
13
14 // Connection à MongoDB
15 mongoose.connect(process.env.DB_URL);
16
17 const db : Connection = mongoose.connection;
18 db.on( eventName: 'error', console.error.bind(console, argArray: 'connection error:'));
19 db.once( eventName: 'open', listener: function() : void { ± Edouard26
20     console.log("Nous sommes connectés à la base de données!");
21 });
22
23 // Modèle d'utilisateur pour Mongoose
24 const UserSchema : Schema<any, Model<...>, {...}, {...}, {...}, DefaultSchemaOptions, {...}, HydratedDocument<...> = new mongoose.Schema( definition: {
25     username: String,
26     password: String,
27 });
28 const UserModel : Model<...> = mongoose.model( name: 'User', UserSchema);
29
30 // Utilisation de la session pour le suivi de l'utilisateur
31 app.use(session( options: {
32     secret: process.env.SESSION_SECRET,
33     resave: false,
34     saveUninitialized: false,
35 }));
36
37
38 app.use(flash());
39
```

```

40
41 passport.use(new LocalStrategy( options: (username, password, done) :void => {
42     UserModel.findOne( filter: { username: username }, projection: (err, user) :any => {
43         if (err) { return done(err); }
44         if (!user) {
45             return done(null, false, { message: 'Incorrect username.' });
46         }
47         if (!bcrypt.compareSync(password, user.password)) {
48             return done(null, false, { message: 'Incorrect password.' });
49         }
50         return done(null, user);
51     });
52 });
53
54 passport.serializeUser( fn: (user, done) => done(null, user.id)); // Edouard26
55 passport.deserializeUser( fn: (id, done) :void => { // Edouard26
56     UserModel.findById(id, projection: (err, user) => done(err, user));
57 });
58 app.use(passport.initialize());
59 app.use(passport.session());
60
61 // Importez le module authRoutes
62 const authRouter :Router = require('./authRoutes')(passport);
63
64 // Créer un utilisateur (enregistrement)
65 app.post( path: '/auth/register', handlers: (req :Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res :Response<ResBody, LocalsObj> ) => { // Edouard26
66     // Logic for registration
67 });
68
69 // Connecter l'utilisateur (connexion)
70 app.post( path: '/auth/login', handlers: (req :Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res :Response<ResBody, LocalsObj> ) => { // Edouard26
71     // Logic for login
72 });
73
74 // Utilisez le routeur
75 app.use('/auth', authRouter);
76
77 // Servir les fichiers statiques de l'application React
78 app.use(express.static(path.join(__dirname, 'dist')));
79
80
81 // Pour toute demande, servir l'index.html
82 app.get('*', (req :Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res :Response<ResBody, LocalsObj> ) :void => { // Edouard26
83     res.sendFile(path.join(__dirname, 'dist', 'index.html'));
84 });
85
86 const PORT :string | number = process.env.PORT || 3001;
87 app.listen(PORT, hostname: () :void => { // Edouard26
88     console.log('Server is running on port ${PORT}');
89 });
90

```

Annexe 12:

Register.js

```
1 import React, { useState } from 'react';
2 import axios from 'axios';
3 import { useHistory } from 'react-router-dom';
4
5 const Register = () => { Show usages  Edouard26
6   const [username :string , setUsername] = useState( initialState: "");
7   const [password :string , setPassword] = useState( initialState: "");
8   const history = useHistory();
9   const handleSubmit = e => { Show usages  Edouard26
10     e.preventDefault();
11
12     axios.post( url: 'http://localhost:3001/auth/register', data: {
13       username: username,
14       password: password
15     }) Promise<AxiosResponse<...>>
16       .then(response :AxiosResponse<any> => {
17         console.log(response);
18         // Si l'enregistrement est réussi, redirigez vers la page de connexion
19         if (response.data.success) {
20           history.push('/login');
21         }
22       }) Promise<void>
23       .catch(error => {
24         console.error('Erreur:', error);
25       });
26   };
27
28   return (
29     <form onSubmit={handleSubmit}>
30       <label>
31         Nom d'utilisateur:
32         <input type="text" value={username} onChange={e :ChangeEvent<HTMLInputElement> => setUsername(e.target.value)} />
33       </label>
34       <label>
35         Mot de passe:
36         <input type="password" value={password} onChange={e :ChangeEvent<HTMLInputElement> => setPassword(e.target.value)} />
37       </label>
38       <button type="submit">S'inscrire</button>
39     </form>
40   );
41 };
42
```

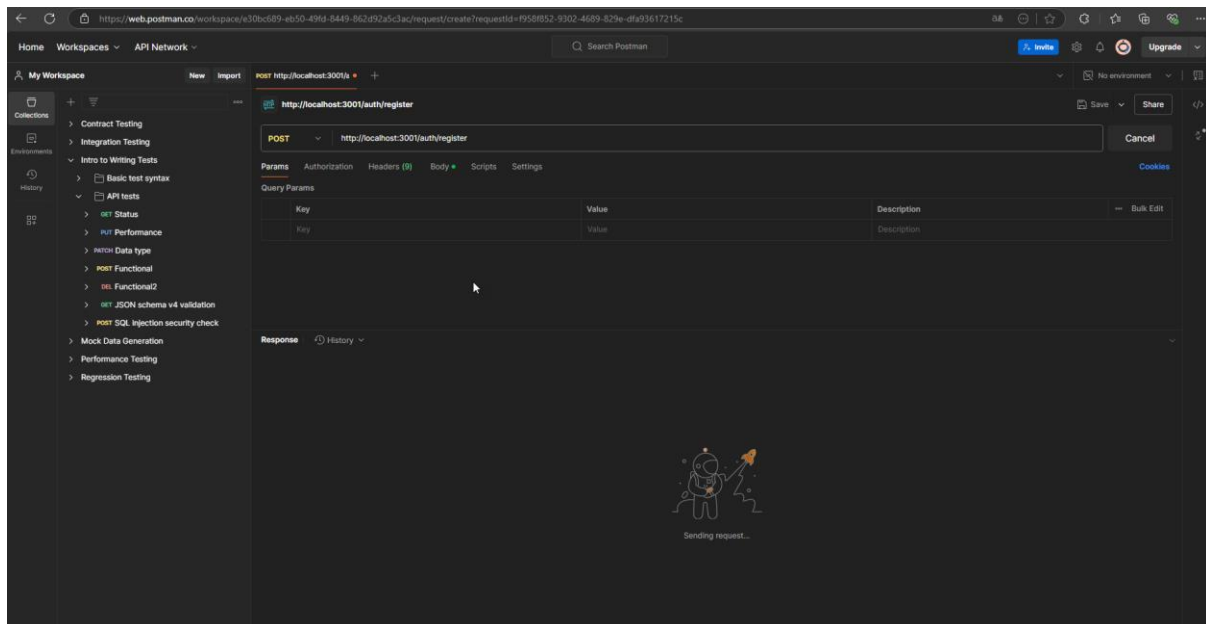
Annexe 13:

Login.js

```
1  > import ...
3
4  const Login = () => { Show usages  ⚡ Edouard26
5    const [username : string , setUsername] = useState( initialState: "");
6    const [password : string , setPassword] = useState( initialState: "");
7    const [token : string , setToken] = useState( initialState: "");
8
9    const handleSubmit = e => { Show usages  ⚡ Edouard26
10      e.preventDefault();
11
12      axios.post( url: 'http://localhost:3001/auth/login', data: {
13        username: username,
14        password: password
15      }) Promise<AxiosResponse<...>>
16        .then(response : AxiosResponse<any> => {
17          console.log(response);
18          if (response.data.token) {
19            setToken(response.data.token);
20          }
21        }) Promise<void>
22        .catch(error => {
23          console.error('Erreur:', error);
24        });
25    };
26
27    return (
28      <form onSubmit={handleSubmit}>
29        <label>
30          Nom d'utilisateur:
31          <input type="text" value={username} onChange={e : ChangeEvent<HTMLInputElement> => setUsername(e.target.value)} />
32        </label>
33        <label>
34          Mot de passe:
35          <input type="password" value={password} onChange={e : ChangeEvent<HTMLInputElement> => setPassword(e.target.value)} />
36        </label>
37        <button type="submit">Login</button>
38      </form>
39    );
40  };
41
42  export default Login; no usages  ⚡ Edouard26
```

Annexe 14:

Test registration avec POSTMAN:



Annexe 15

Tableau de bord AWS

ML-Explorer tableau de bord ☆ Informations Actions Gérer les ressources Réinitialiser la mise en page par défaut + Ajouter des widgets

Résumé de l'application Informations

Nom
ML-Explorer

Description
platform that is aimed at an audience other than machine learning students or researchers. The goal is to offer a popularized version for anyone interested in machine learning without skills.

Région
eu-west-3

Clé d'identification de l'application
awsApplication

Valeur d'identification de l'application

Coût et utilisation informations

Coûts du mois en cours
0,00 \$US

Ventilation des coûts
Aucune donnée sur les coûts n'est disponible.

Coûts prévisionnels de fin de mois
Données non disponibles

Opportunités Savings
Activer le hub d'optimisation des coûts

Annexe 16:

Carte Trello "Documentation et Ressources »

The screenshot shows a Trello card interface. At the top, the card title is "Travail sur la Documentation et les Ressources" with a sub-label "dans la liste CONCEPTION". Below the title, there's a "Notifications" section with a "Suivre" button. The main content area is titled "Description" and contains two sections: "Exercice sur Kaggle:" with a bulleted list of tasks (Algorithm of regression, Training/Validation of the Model, TITANIC competition) and "Hugging Face:" with a bulleted list (Research on fine tuning, Dataset download, Exchange on Discord). To the right of the description is a "Modifier" button. Below the description is a "Checklist" section with a progress bar at 100% and a list of tasks, all of which are checked. To the right of the checklist are buttons for "Masquer les tâches cochées" and "Supprimer". Below the checklist is an "Ajouter un élément" button. On the right side of the card, there's a sidebar with various options: "Rejoindre", "Membres", "Étiquettes", "Checklist", "Pièce jointe", "Emplacement", "Champs personnalisés", "Power-Ups" (with an "Ajouter des Power-ups" button), "Automatisation" (with an "Ajouter un bouton" button), and "Actions" (with buttons for "Déplacer", "Copier", "Modèle" (checked), "Masquer dans la liste", "Supprimer", and "Partager"). At the bottom, there's an "Activité" section with a comment input field and an "Afficher les détails" button.

Travail sur la Documentation et les Ressources
dans la liste CONCEPTION

Notifications
Suivre

Description Modifier

Exercice sur Kaggle:

- Algorithme de régression
- Entraînement / Validation du Modèle (librairie scikit-learn)
- Compétition d'entraînement TITANIC

Hugging Face:

- Recherche sur le fine tuning des modèles d'IA
- Téléchargement de Dataset
- Echange via Discord avec la communauté Hugging Face

☒ Checklist Masquer les tâches cochées Supprimer

100%

- ☒ Algorithme de régression
- ☒ Entraînement / Validation du Modèle (librairie scikit-learn)
- ☒ Compétition d'entraînement TITANIC
- ☒ Recherche sur le fine tuning des modèles d'IA
- ☒ Téléchargement de Dataset
- ☒ Echange via Discord avec la communauté Hugging Face

Ajouter un élément

Activité Afficher les détails

EH Écrivez un commentaire...

Rejoindre
Membres
Étiquettes
Checklist
Pièce jointe
Emplacement
Champs personnalisés

Power-Ups
Ajouter des Power-ups

Automatisation
Ajouter un bouton

Actions
Déplacer
Copier
Modèle
Masquer dans la liste
Supprimer
Partager

Annexe 17 :

Carte Trello « Maquette de l'Application »

The screenshot shows a Trello card titled "Maquette de l'Application" within a board named "CONCEPTION". The card is in "Suivie" (Followed) status. It features a "Description" section with a list of tasks: "chemin de l'application", "interface utilisateur", "structure du serveur & DB", and "gestion des écrans". A "Checklist" section shows these same tasks as completed (100%). The right sidebar includes options like "Rejoindre", "Membres", "Étiquettes", "Checklist", "Pièce jointe", "Emplacement", "Champs personnalisés", "Power-Ups", "Automatisation", and "Actions".

Maquette de l'Application
dans la liste **CONCEPTION**

Notifications
Suivie ✓

Description Modifier

Réalisation des différents diagrammes nécessaires à la réalisation de mon application:

- chemin de l'application
- interface utilisateur
- structure du serveur & DB
- gestion des écrans

Checklist Masquer les tâches cochées Supprimer

100%

- ✓ chemin de l'application
- ✓ interface-utilisateur (UI)
- ✓ structure-du-serveur-et-DB
- ✓ gestion-des-écrans

Ajouter un élément

Rejoindre

Membres

Étiquettes

Checklist

Pièce jointe

Emplacement

Champs personnalisés

Power-Ups
+ Ajouter des Power-ups

Automatisation ⓘ
+ Ajouter un bouton

Actions
→ Déplacer

Annexe 18

Carte Trello « Réalisation du Code »

Réalisation du Code

dans la liste

CONCEPTION

Notifications

Suivie

Description

Modifier

Créer tout les composants nécessaires au fonctionnement de mon application.

front-end /UI Components DataTable , DataDisplay, DataCard, ColumnDetails, ModelCard, Result, Dataset, TrainModel, SaveModel, AddFeature, AnimationDescription, ColumnDataType, Column Inspector

JS components: App.js, index.js, train.js, redux.js

back-end / Implémentation du serveur node

Server Components: Server.js, Login.js, Register.js, AuthRoutes.js

configuration des routes et contrôleurs: authRoutes.js

datasets

Checklist

Masquer les tâches cochées

Supprimer

100%

UI Components DataTable , DataDisplay, DataCard, ColumnDetails

UI Components ModelCard, Result, Dataset, TrainModel, SaveModel

UI Components AddFeature, AnimationDescription, ColumnDataType, Column Inspector

JS Components App.js, index.js, train.js, redux.js

Server Components: Server.js, Login.js, Register.js, AuthRoutes.js

datasets: .csv files, .jpg files, .json files

Rejoindre

Membres

Étiquettes

Checklist

Pièce jointe

Emplacement

Champs personnalisés

Power-Ups

Ajouter des Power-ups

Automatisation

Ajouter un bouton

Actions

Déplacer

Copier

Modèle

Masquer dans la liste

Supprimer

